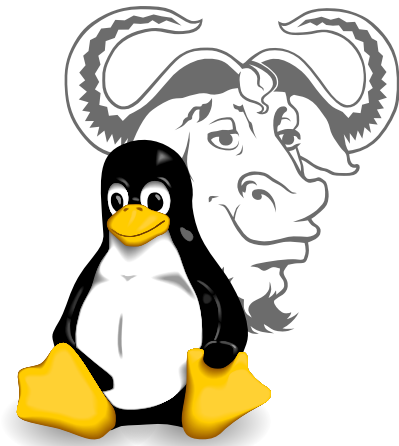


UD05.- Administración de un Sistema Operativo Libre II

Continuamos en este tema ampliando apartados sobre la administración y seguridad del sistema operativo.



Larry Ewing (Free Art License)

<[https://
commons.wikimedia.org/
wiki/File:Gnulinux.svg](https://commons.wikimedia.org/wiki/File:Gnulinux.svg)>

1.- Gestión de usuarios y grupos



Caso práctico



Nuria Barroso
(Elaboración
propia)

Desde el servidor de Linux que han instalado en la empresa, Iván va a tener que hacer labores administrativas, por ejemplo, va a tener que dar de alta a un grupo de usuarios y, además, tiene que hacer que pertenezcan al mismo grupo, para luego facilitar el poner o quitar permisos a todo el grupo. Como el servidor sólo tiene entorno de texto, esta tarea la tiene que realizar mediante comandos.

1.1.- Comandos sobre permisos

Ya sabrás que los permisos más frecuentes que puede tener un fichero o directorio son: de lectura (r), escritura (w) y ejecución (x).

- **Permiso de lectura:** Solamente se permite la lectura, no la modificación. En el caso de carpetas, no se permite la creación de archivos nuevos ni de subcarpetas.
- **Permiso de escritura:** Se permite la lectura y la modificación. En caso de carpetas, se pueden crear archivos y subcarpetas.
- **Permiso de ejecución:** Este permiso solamente tiene sentido en caso de archivos ejecutables (programas) y carpetas. Si este permiso está activo para el usuario concreto, éste podrá ejecutar el programa.

Cuando ejecutas la orden `ls -l` en el terminal, la información de una de sus líneas cualquiera puede ser:

```
-rwxr-xr-x 1 primero primero 4096 2011-05-12 fichero
```

Como puedes observar, los permisos están aparecen dentro de los datos dentro de la primera columna. Si te fijas, la primera columna muestra la siguiente información: `-rwxr-xr-x`

Cuando un objeto (fichero o directorio) tiene un permiso asignado, aparece en su lugar el carácter que representa al permiso (r, w o x). *Cuando un fichero o directorio carece de un permiso aparece en su lugar un guion.*

El primer carácter de esta columna hace referencia al tipo de objeto del cual se trata. Es decir, si es un directorio, fichero, socket, etc. Los ficheros normales se representan con el símbolo '-'. Si fuera un directorio aparecería el carácter 'd', si se tratase de un fichero de tipo socket (especiales) aparecería una 's', si es un enlace aparecería una 'l', etc.

Desde el segundo carácter, de la primera columna, hasta el final de la misma, aparecen representados los permisos para el usuario propietario del fichero, para los usuarios que pertenecen al grupo propietario del fichero o directorio y los permisos para el resto de usuarios del sistema.

- **Propietario,** son los permisos del propietario del directorio o fichero.
- **Grupo,** son permisos sobre el directorio o fichero del grupo que se indica a continuación.
- **Otros,** son permisos del resto de usuarios sobre el directorio o el fichero.

Para cambiar los permisos puedes usar los siguientes comandos:

- **chown** usuario fich (chown es acrónimo de Change Owner): Cambia el propietario de un fichero.
- **chgrp** grupo fich (chgrp acrónimo de change group): Cambia el grupo al que pertenece un fichero.
- **chmod** permisos fich chmod (acrónimo de change mode): Cambia los permisos de acceso del fichero añadiendo o quitando, o según se especifica en la cadena de permisos.
 - -u[+/-r][+/-w][+/-x] Añade o quita el permiso especificado para el usuario.
 - -g[+/-r][+/-w][+/-x] Añade o quita el permiso especificado para el grupo.

Modificadores y argumentos para la orden chmod.

Quién	u: es el usuario creador.	Acción	r: permiso de lectura (4 €
	g: es el grupo al que pertenece el usuario.		w: permiso de escritura (2
	o: es el resto de usuarios del equipo.		x: permiso de ejecución (1
	a: todos los usuarios del sistema		

Otra forma de modificar los permisos es mediante un **número octal** de tres cifras por cada grupo de permisos, este número surge de realizar la suma de los permisos que se les quieren asignar de acuerdo a los valores. r=4, w=2 y x=1.

Por ejemplo, si quieres dar al fichero **fichero1** todos los permisos a todos los usuarios, es decir, al usuario creador, al grupo al que pertenece este usuario y al resto de usuarios del sistema, escribe en el terminal una de las siguientes opciones:

- `chmod ugo + rwx fichero1.`
- `chmod 777 fichero1.`

Puedes ver sus nuevos permisos escribiendo en el terminal: `ls -l fichero1`

En los apartados siguientes vamos a ver como asignar o quitar permisos a los ficheros o directorios utilizando el comando chmod. Como hemos visto anteriormente los permisos se pueden asignar o eliminar utilizando una representación numérica o mediante letras.



Autoevaluación

El archivo permisos.txt tiene 751 permisos:

- ☐ El usuario, el grupo y el resto de usuarios tienen permiso de lectura solamente.
- ☐ El usuario tiene todos los permisos, el grupo de lectura y ejecución y el resto de

usuarios sólo ejecución.

- ☐ Tienen todos solamente permisos de ejecución.

Incorrecta, ya que $u=7 = 4+2+1 = r+w+x$, es decir, el usuario tiene permiso de lectura, escritura y ejecución; $g=5 = 4+1 = r+x$, es decir, el grupo tiene permiso de lectura y de ejecución; $o=1=x$, es decir, el resto de usuarios sólo tiene permiso de ejecución.

Muy bien, has captado la idea. $u=7 = 4+2+1 = r+w+x$, $g=5 = 4+1 = r+x$, $o=1=x$.

No es la respuesta correcta. Ya que $u=7 = 4+2+1 = r+w+x$, $g=5 = 4+1 = r+x$, $o=1 = x$.

Solución

1. Incorrecto
 2. Opción correcta
 3. Incorrecto
-

1.1.1. Modificación de permisos mediante representación numérica

Los permisos se asignan, se quitan o se modifican de la misma forma a los ficheros y a los directorios. En el momento de modificar los permisos solo debemos indicar los permisos a asignar y el nombre del objeto al cual queremos asignar o quitar permisos. Podemos modificar los permisos de un objeto (fichero o directorio) para asignarle los permisos necesarios o para quitarle permisos.

El comando que utilizaremos para modificar los permisos de un objeto es el comando `chmod`. Si queremos modificar los permisos de un objeto utilizando la representación numérica, hay que especificar los permisos utilizando un *número de tres dígitos utilizando la representación octal*. Este número podrá tener un valor comprendido entre 000 y 777. Cada uno de los dígitos que forman el número tiene un significado. En este caso, la sintaxis del comando `chmod` es:

```
chmod permisos fichero_o_directorio
```

Siempre que utilicemos la representación numérica deberemos utilizar un número que tenga tres dígitos. Cada dígito podrá tener un valor entre 0 y 7 porque son los diferentes valores que existen en el sistema de numeración octal.



Debes conocer

Siempre que utilicemos la representación numérica deberemos utilizar un número que tenga tres dígitos. Cada dígito podrá tener un valor entre 0 y 7 porque son los diferentes valores que existen en el sistema de numeración octal.

El número que utilizamos para asignar o quitar permisos, representa 3 tipos de permisos: permiso de lectura (r), escritura(w) y ejecución(x) en este orden. Para obtener un valor concreto utilizaremos el sistema binario. Utilizaremos 1 para indicar que **SI** tiene permiso (asignar) y 0 para quitar permisos (denegar), es decir, que **NO** tiene permisos.

Cada dígito significa los permisos para un grupo de usuarios diferentes.

El **primer dígito** significa los permisos que se le van a asignar al fichero para el propietario del fichero.

El **segundo dígito** significa los permisos para los usuarios que pertenecen al **grupo de usuarios** al cual pertenece el fichero o directorio

El **tercer dígito** son los permisos para el **resto de usuario** que no son ni el usuario propietario del fichero o directorio ni los usuarios que pertenezcan al grupo al cual pertenece el fichero o directorio.



Agustín Nieto Espino (Elaboración propia)

En el ejemplo se puede ver, que al usuario propietario le corresponden los permisos de lectura, escritura y ejecución sobre el objeto. Le corresponde el primer 7 de los números indicados. Si convertimos 7 a binario se corresponde con el número 111. El primer 1 (empezando por la izquierda) indica que, si tiene permiso de lectura, el segundo uno indica que si tiene permiso de escritura y el tercer uno indica que si tiene permiso de ejecución.

A los usuarios miembro del grupo al cual pertenece el objeto también le corresponden los permisos de lectura, escritura y ejecución, el 7.

Al resto de usuarios, solo le corresponden los permisos de lectura y ejecución, por lo tanto, no podrán modificar el fichero, pero si acceder al mismo. En el ejemplo, al resto de usuarios del sistema (otros), le corresponde el valor 5. Si convertimos 5 al binario obtendremos el número 101 (4+1). Por lo tanto, el primer uno indica que el resto de usuarios del sistema si tiene permisos de lectura, el cero que aparece a continuación indica que no tiene permisos de escritura y el segundo uno, indica que si tiene permiso de ejecución.



Recomendación

Para comprender correctamente la asignación o eliminación de permisos utilizando este método de representación, es necesario saber cómo se convierte un número de octal o decimal a binario y cómo convertir un número de binario a octal o decimal. Puedes repasar los contenidos vistos sobre sistemas de numeración en el módulo de Montaje y mantenimiento de equipos.



Ejercicio Resuelto

Asigna únicamente permisos de lectura a un fichero, llamado practica1.sh pero solamente para el dueño del fichero y los usuarios del mismo grupo.

```
chmod 440 practica1.sh
```

Asigna solo permisos de lectura y escritura para el propietario y para los usuarios que pertenecen al mismo grupo de usuarios a los cuales pertenece el fichero practica2.sh

```
chmod 660 practica2.sh
```

Asigna permisos de lectura, escritura y ejecución para el propietario, lectura y escritura para los miembros del grupo al cual pertenece el siguiente fichero y permiso de lectura para el resto de usuarios sobre el fichero practica3.sh

```
chmod 764 practica3.sh
```

1.1.2. Modificación de permisos mediante representación de caracteres

Otra forma de cambiar los permisos es especificar los tipos de acceso y los tipos de usuarios mediante una letra identificativa de cada uno de ellos.

Para identificar los tipos de usuarios:

IDENTIFICADOR	TIPOS DE USUARIOS
u	Usuario propietario
g	Grupo propietario.
o	Resto de usuarios.
a	Todos los usuarios.

Para identificar los tipos de permisos:

IDENTIFICADOR	TIPOS DE PERMISOS
r	Permiso de lectura
w	Permiso de escritura
x	Permiso de ejecución

Para asignar permisos utilizaremos el símbolo +

Para quitar o eliminar permisos utilizaremos el símbolo -

Ejemplo_1

Para dar permisos de escritura y ejecución a todos los usuarios sobre el fichero practica2.sh, en el terminal escribiríamos la orden:

```
chmod a+wx practica2.sh
```

Ejemplo_2

Para quitar los permisos de escritura y ejecución a todos los usuarios sobre el fichero practica2.sh, en el terminal escribiríamos la orden:

```
chmod a-wx practica2.sh
```

1.1.3. Permisos especiales

Cada archivo puede tener 3 permisos especiales que pueden alterar la forma en la que funciona un directorio o cómo se ejecuta un programa.

Sticky Bit (Bit pegajoso)

Se aplica solo a directorios y evita que los usuarios eliminen o cambien el nombre de un archivo o directorio a menos que sean propietarios de ese archivo o directorio.

Tiene el valor octal 1 y en modo simbólico está representado por una `t` dentro de los permisos de "otros".

- `t`: con permiso de ejecución para otros
- `T`: sin permiso de ejecución para otros

Por ejemplo:

```
ls -ld /tmp
```

```
drwxrwxrwt 13 root root 4096 ene 24 14:28 /tmp
```

Para el caso anterior del directorio `tmp`, todos los usuarios tienen permiso de lectura y escritura, ya que los procesos que hayan iniciado pueden tener archivos temporales ahí. Poniendo en directorio con el sticky bit (`t`) solo puede borrar cada archivo el usuario propietario.

En el modo numérico los permisos especiales se especifican usando una notación de 4 dígitos, con el primer dígito representando el permiso especial para actuar.

Por ejemplo para establecer el sticky bit de `Another_Directory` en modo numérico con permisos 755, el comando sería:

```
chmod 1755 Another_Directory
```

```
ls -ld Another_Directory
```

```
drwxr-xr-t 2 profe profe 4,0K ene 24 18:46 Another_Directory
```

Set GID

Se puede aplicar a archivos ejecutables o directorios:

- En archivos ejecutables: ejecuta el archivo con los privilegios del grupo
- En directorios: hace que cada archivo o directorio creado debajo herede el grupo del directorio principal

Tiene el valor **octal 2** y en modo simbólico está representado por una **s** dentro de los permisos de "grupo".

```
ls -ld /tmp
```

```
-rwxr-sr-x 1 profe profe 33 ene 24 10:36 test.sh
```

Para agregar permisos SGID a un archivo en **modo simbólico**, el comando sería:

```
chmod g+s test.sh
```

```
ls -l test.sh
```

```
-rwxr-sr-x 1 profe root 33 ene 24 10:36 test.sh
```

Set UID

Solo se aplica a archivos ejecutables e implica que el proceso se ejecutará con los permisos del propietario del archivo

Tiene el valor **octal 4** y en modo simbólico está representado por una **s** dentro de los permisos de "usuario".

```
ls -l test.sh
```

```
-rwsr-xr-x 1 profe profe 33 ene 24 10:46 test
```

Combinación de permisos especiales

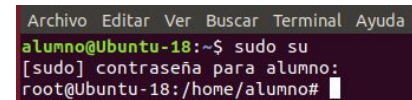
Se pueden combinar múltiples permisos especiales en un parámetro agregándolos juntos en octal en el primer dígito (4+2+1) de la notación en 4 dígitos anteriormente vista (**especiales** usuario grupo otros):

```
chmod 6755 test.sh
```

```
ls -lh test.sh
```

```
-rwsr-sr--x 1 profe profe 66 ene 17:29 tst.sh
```

1.2.- Comandos de gestión de usuarios



```
Archivo Editar Ver Buscar Terminal Ayuda
alumno@Ubuntu-18:~$ sudo su
[sudo] contraseña para alumno:
root@Ubuntu-18:/home/alumno#
```

Agustín Nieto Espino (Elaboración propia)

Vas a ver una serie de comandos muy usuales en la administración básica de un equipo. Para ejecutar estos comandos necesitaremos permisos de administrador o bien perteneciente al grupo de administradores.

- **useradd usuario** (acrónimo de user add): añade usuarios. Nos permite crear un usuario nuevo en el sistema. Si le pasamos el parámetro **--group** nos permite crear el usuario y añadir a un grupo de usuarios.
- **useradd --group nombre_grupo_usuario usuario**: crea el usuario *usuario* perteneciente al grupo indicado. Es requisito que el grupo exista previamente.

Si le pasamos el parámetro **-d** seguido de una ruta, estaremos asignando dicha ruta como directorio de trabajo al usuario que vamos a crear. Por ejemplo, si queremos crear la usuaria rocio y asociarle el directorio `/home/rocio`, tendremos que escribir la orden: `useradd -d /home/rocio rocio`. La ruta debe de ser creada previamente.

Si queremos asignarle un intérprete de comandos, por ejemplo, `bash`, debemos de pasarle el parámetro **-s** seguido de la ruta donde se encuentre el intérprete de órdenes que queramos utilizar. Suponiendo que la ruta `/home/rocio` existe y queramos dar de alta en el sistema a la usuaria rocio para que utilice dicha ruta como directorio de trabajo y, además, que utilice el intérprete `bash` cuando inicie sesión, escribiríamos en el terminal la orden:

```
useradd -d /home/rocio -s /bin/bash rocio
```

Cuando damos de alta un usuario en el sistema con el comando `useradd`, éste por defecto, no le asocia ninguna contraseña al usuario. Para poder utilizar el usuario necesitaremos asignarle una contraseña previamente. Esto lo podemos hacer una vez creado el usuario/a con el comando `passwd` o bien pasándole el parámetro **-p** en el momento de crear el usuario seguido de la contraseña. *La contraseña se encontrará escrita en modo texto con lo cual puede suponer un problema de seguridad.* Por esta razón, te recomendamos que, si das de alta un usuario con este comando, utilices a continuación el comando `passwd` para asignarle una contraseña.

- **userdel usuario** (acrónimo de usr delete): elimina usuarios. Si le pasamos el parámetro **-r**, borra también toda la información de su directorio personal.
- **adduser nombre_usuario**: añade un usuario nuevo al sistema, creando un *directorio de trabajo con el mismo nombre de usuario dentro de la ruta /home* y le asigna una

contraseña. Más cómodo de utilizar que el comando `useradd*`.

- **passwd** `usuario`: cambia la contraseña del usuario indicado
- **groupadd** `nuevogrupo`: para añadir un nuevo grupo de usuarios
- **groupdel** `nombre_del_grupo`: para eliminar un grupo.
- **delgroup** `nombre_usuario` `nombre_grupo`: nos permite eliminar a un usuario de un grupo de usuarios.
- **usermod**: cambia varios atributos de usuarios. Todo usuario puede pertenecer a un solo grupo principal y a varios secundarios. Si le pasamos los siguientes parámetros:

-g `nombre_grupo` `usuario`: cambia el grupo principal de un usuario.

-G `nombre_grupo` `usuario`: nos permite añadir un usuario un grupo secundario.

- **id**: muestra el nombre y número de usuario y grupo/s al que pertenece.
- **whoami**: muestra el nombre de usuario.
- **who**: muestra los nombres de los usuarios conectados al sistema, sus terminales y la fecha/hora de acceso. En el caso de utilizar la versión desktop de Ubuntu, mostrará los usuarios que han iniciado sesión desde el entorno gráfico.

Por ejemplo, para crear un nuevo usuario, como es una tarea de administrador, escribe en la terminal: `sudo useradd usuarioNuevo`

Puedes comprobar que se ha creado un nuevo usuario desde el entorno gráfico.

En Mostrar Aplicaciones > Usuarios y Grupos, o bien, viendo el contenido del fichero `passwd`, escribiendo en la terminal: `cat /etc/passwd`

Comprueba que en la última línea hay un nuevo usuario, además, que como UID asignado tiene uno mayor que 1000, ya que es un usuario normal. Para ver el número UID también puedes escribir en la terminal: `id usuarioNuevo`

Toda la información sobre usuarios, grupos y contraseñas se guarda en los archivos:

- `/etc/passwd`: Información sobre usuarios.
- `/etc/group`: Información sobre grupos.
- `/etc/shadow`: contraseñas cifradas.

En estos ficheros se almacena la información línea a línea (cada una es un usuario o un grupo).

PROBLEMA DE SEGURIDAD CON LA CONTRASEÑA DE

USUARIO CREADO

Si creas las contraseñas al crear el usuario con el parámetro **-p** seguido de la contraseña, *la contraseña se encontrará escrita en modo texto*. Por esta razón, se recomienda que, si das de alta un usuario con este comando, utilices a continuación el comando **passwd** para asignarle una contraseña.

useradd vs adduser

useradd es un comando que ejecuta un binario del sistema, mientras que **adduser** es un script en perl que utiliza el binario **useradd**.

La mayor ventaja del comando **adduser** es que *crea el directorio home (/home/usuario/)* del usuario de manera automática, cosa que *no hace useradd* (hay que usar la opción **-m**). Sin embargo, como no es un comando del core de GNU/Linux, es posible que no funcione bien en todas las distribuciones que uses.

La ventaja de **useradd** es que funciona igual en todas las distribuciones.



Debes conocer

En el siguiente vídeo, vas a ver cómo crear y eliminar usuarios desde un terminal:

Gestión de usuarios

La reproducción del vídeo se ha interrumpido por un error de red.

11:07 / 11:07

Agustín Nieto Espino. **Descripción textual alternativa para el vídeo "Gestión de usuarios"**. (Elaboración propia)



Debes conocer

A través del siguiente vídeo puedes ver cómo crear y administrar grupos de usuarios desde el terminal:

Gestión de grupos

Gestión de grupos

Profesor: Agustín Nieto Espino

0:00 / 6:11

Agustín Nieto Espino. Descripción textual alternativa para el vídeo "Gestión de grupos". (Elaboración propia)

2.- Gestión de procesos

En todo sistema operativo hablamos de procesos a cualquier programa, comando o aplicación que se ejecuta. Vamos a ver con más detalle qué son los procesos en Linux y cómo identificarlos. También aprenderemos a gestionarlos y a manipularlos en función de nuestras necesidades.



2.1.- Qué son los procesos

Tal vez no somos conscientes, pero cuando ejecutamos un comando, herramienta, aplicación, software, etc, en Linux, se genera (como mínimo) un nuevo proceso.

Un proceso almacena información importante como ejemplo:

- Sus propios permisos
- Datos
- Instrucciones
- Identificador de proceso (PID)
- ...

Durante el tiempo que se ejecute el proceso hará uso de los recursos del sistema, la CPU para ejecutar sus instrucciones propias y de la memoria RAM para almacenar datos. Es evidente que también utiliza los archivos propietarios del sistema, además de interactuar con los dispositivos de almacenamiento.

De la información que almacena un proceso nos interesa en este momento el PID. Linux asigna un número único de cinco dígitos a cada proceso. Al número generado se le conoce como PID (número de identificación de proceso) y es una manera excelente para identificar los procesos, ya que no es posible tener dos PID iguales, cada proceso tiene su número exclusivo hasta que muera.

Para saber el PID de un proceso en concreto tenemos que ejecutar el comando `pgrep` proceso:

```
$ pgrep firefox  
865
```

o también podemos averiguar los procesos del usuario actual con:

```
$ ps  
  PID TTY          TIME CMD  
 2786 pts/0    00:00:00 bash  
13023 pts/0    00:00:00 ps
```

Tipos de procesos

En Linux podemos clasificar los procesos que se están ejecutando de diferentes maneras:

Hay cuatro tipos de procesos comunes:

- El **proceso padre o principal** es el que inicia la ejecución. Todos los procesos, excepto el proceso 0, tienen un proceso principal.
- **Proceso hijo** es creado por un proceso padre.
- **Proceso huérfano** continúa ejecutándose después que su proceso principal (padre) ha sido terminado o finalizado.
- **Proceso Daemon** siempre se crea desde un proceso secundario y luego termina.
- **Proceso Zombie** existe en la tabla de procesos, aunque haya terminado.

El proceso huérfano es un proceso que todavía se está ejecutando y su proceso principal ha terminado, estos no pueden convertirse en procesos zombies.

Según el plano de ejecución

Procesos en primer plano

Cuando el usuario está interactuando con un proceso, se dice que este proceso está en primer plano. Los «**Frontend processes**» son procesos interactivos en primer plano iniciados o lanzados por algún usuario. Un ejemplo claro son los comandos que ejecutamos en la **terminal**, siempre comienzan en primer plano.

Otros procesos 'frontend' son los que se inician desde interfaces gráficas, y permiten interactuar con ellos.

Sólo un proceso puede estar en primer plano en un momento dado. Por el contrario, en segundo plano pueden existir varios procesos a la vez. Por otro lado, un proceso puede pasarse de primer plano a segundo plano, y viceversa, cuantas veces se desee.

Procesos en segundo plano

Se conocen como «**Background processes**» a los procesos en segundo plano, es decir, a los procesos automáticos o no interactivos. Se caracterizan por:

- Se inician automáticamente sin intervención del usuario y no requieren ninguna respuesta.
- Son independientes y, una vez que son creados, harán su trabajo sin ninguna

interacción con quien los creo.

Un ejemplo son los **demonios** <<https://medium.com/jmtorres/servicios-y-demonios-en-linux-a424366336ac>> o los **servicios del sistema** <<https://www.hostinger.es/tutoriales/administrar-y-listar-servicios-en-linux/>> .

Las principales ACCIONES que pueden realizarse sobre procesos desde la línea de comandos son:

- **Mostrar información sobre los procesos:**

El comando **ps** (abreviatura de Process Status) muestra un listado con el estado de los procesos.

El comando **pstree** es una variante de **ps** en la que los procesos se muestran en forma de árbol.

El comando **jobs** se utiliza para mostrar los procesos que se están ejecutando en primer y en segundo plano.

- **Cambiar el estado de los procesos.**

El comando **kill** permite enviar señales a los procesos para cambiar su estado. La señal por defecto de este comando es terminar (matar) el proceso, de ahí su nombre.

- **Cambiar la prioridad de los procesos.**

El comando **nice** se emplea para cambiar la prioridad de un proceso cuando se inicia su ejecución. Por defecto el valor de la prioridad es 0. Sólo los usuarios **root** tienen privilegios para asignar prioridades negativas.

El comando **renice** permite cambiar la prioridad de un proceso sin necesidad de detenerlo.

- **Cambios de plano de los procesos.**

El operador **&** se utiliza para ejecutar un proceso en segundo plano directamente.

El comando **nohup** se usa para ejecutar un comando y que no termine su ejecución aunque se cierre la terminal.

Cuando un proceso está detenido, se puede utilizar los comandos **fg** y **bg** para ponerlo en primer plano o en segundo plano, respectivamente.

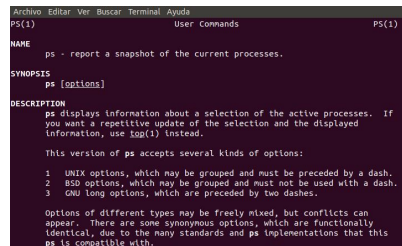


Debes conocer

En Linux cada proceso tiene un proceso padre, pero:

- “init” es el primer proceso que crea el kernel Linux cuando inicia el sistema (boot).
 - Todos los procesos son hijos de init (de forma directa o indirecta).
 - El proceso init **no puede ser matado** (kill), excepto cuando se apaga el sistema.
 - El proceso init siempre tiene el PID = 1
-

2.2.- Comandos de gestión de procesos



Agustín Nieto Espino

(Elaboración propia)

Tendrás la oportunidad de administrar y gestionar procesos en ejecución, mediante comandos específicos para su tratamiento.

Como aplicación más interesante de las señales se estudiará la sincronización de procesos. Con los siguientes comandos podrá ver, cancelar y priorizar procesos en ejecución.

- **ps** (acrónimo de process status): muestra la lista de procesos con sus PID (Identificación del proceso) y nombre. Al ejecutarlo muestra los procesos que ha sido creado desde el terminal actual.
- Algunos de los parámetros que admite son:
 - **-a** Muestra también procesos de otros usuarios.
 - **-x** Muestra también procesos sin terminal de control (daemons).
 - **-u** usuario Muestra los procesos de un usuario específico.
 - **-e** Muestra todos los procesos.
 - **-F** muestra toda la información de cada proceso.
 - **-l** Con todos los detalles (S: estado (S - Sleep, R - Running, T - Stopped), UID: Identificación del usuario que ha lanzado el proceso, PID: Identificación del proceso PPID: identificación del padre del proceso, valor de nice, SZ: uso de memoria y CPU, etc.).
- **kill**: Termina procesos.
- **top**: Muestra en tiempo real la situación de los procesos.

Por ejemplo, si escribes en la terminal, la orden: **ps**

Obtendrás información sobre el PID, TTY, TIME y CMD, ¿y qué es eso?, pues verás:

- **PID**: Cada proceso obtiene un número de identificación único, este es el PID.
- **PPID**: Se muestra el PID de su proceso padre.
- **TTY**: Es la terminal asociada al proceso, también puede ser otro dispositivo de entrada.
- **TIME**: El tiempo que el proceso ha estado en ejecución.

- **CMD:** El nombre del comando que está siendo ejecutado.

Si escribes en un terminal, la orden: **ps -eF**

Verás todos los procesos que se están ejecutando junto con toda su información.



Autoevaluación

Relaciona los comandos con lo que muestran, escribiendo el número asociado a lo que hace el comando en el hueco correspondiente.

Comando Relación Resultado

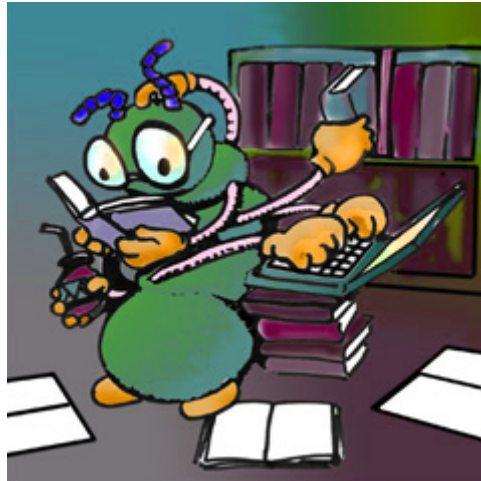
useradd	☐	1.- Lista los procesos.
ps	☐	2.- Muestra los usuarios.
who	☐	3.- Mata procesos.
kill	☐	4.- Añade usuarios.

Son algunos de los comandos más usados de administración básica.

2.3.- Comandos para mostrar información de los procesos

En este apartado, vamos a ver el funcionamiento de los comandos, que se explican brevemente de nuevo a continuación:

- El comando **ps** (abreviatura de Process Status) muestra un listado con el estado de los procesos.
- El comando **ps tree** es una variante de **ps** en la que los procesos se muestran en forma de árbol.
- El comando **jobs** se utiliza para mostrar los procesos que se están ejecutando en primer y en segundo plano.



Stevetigo (CC BY-SA) <<https://commons.wikimedia.org/wiki/File:Wikipedesketch.png>>

2.3.1. Comando ps

Este comando obtiene la información que nos muestra por pantalla del directorio `/proc`.

La tabla que vemos a continuación explica en qué consiste cada una de las columnas que muestra el comando en pantalla:

Columna	Descripción
p o PID	Process ID, número único o de identificación del proceso.
P o PPID	Parent Process ID, padre del proceso
U o UID	User ID, usuario propietario del proceso
t o TT o TTY	Terminal asociada al proceso, si no hay terminal aparece entonces un '?'
T o TIME	Tiempo de uso de cpu acumulado por el proceso
c o CMD	El nombre del programa o comando que inició el proceso
RSS	Resident Size, tamaño de la parte residente en memoria en kilobytes
SZ o SIZE	Tamaño virtual de la imagen del proceso
NI	Nice, valor nice (prioridad) del proceso, un número positivo significa menos tiempo de procesador y negativo más tiempo (-19 a 19)
C o PCPU	Porcentaje de cpu utilizado por el proceso
STIME	Starting Time, hora de inicio del proceso
S o STAT	Status del proceso, estos pueden ser los siguientes • R runnable, en ejecución, corriendo o ejecutándose • S sleeping, proceso en ejecución pero sin actividad por el momento, o esperando por algún evento para continuar • T sTopped, proceso detenido totalmente, pero puede ser reiniciado • Z zombie, difunto, proceso que por alguna razón no terminó de manera correcta, no debe haber procesos zombies • D uninterruptible sleep, son procesos generalmente asociados a acciones de IO del sistema • X dead, muerto, proceso terminado pero que sigue apareciendo, igual que los Z no deberían verse nunca

Veamos varios ejemplos de ejecución de este comando:

Comando: ps

Muestra por pantalla información de los procesos que se están ejecutando en el sistema del usuario actual:

```
profe@profe-vm:~$ ps
  PID TTY          TIME CMD
 2786 pts/0    00:00:00 bash
15263 pts/0    00:00:00 ps
profe@profe-vm:~$
```

En este ejemplo, podemos observar que los procesos que están ejecutándose son el `bash` (el intérprete de comandos) y el mismo comando `ps`.

Aunque ya se ha indicado en la tabla anterior el significado de todas las columnas que muestra este comando, para este primer ejemplo, las columnas en negrita quieren decir que:

- La primera columna es el **PID** o **identificador de proceso**. Cada proceso tiene un asociado identificador que es único, es decir que no puede haber dos procesos con el mismo identificador.
- La segunda columna nos informa del **terminal** en el que se está ejecutando el proceso.
- La tercera columna indica el **tiempo total** que ha estado ejecutándose el proceso.
- La cuarta columna es el **nombre del proceso**.

Comando: ps -f

En este ejemplo, se muestra información de forma extendida, añadiendo:

- Identificador de proceso padre **PPID**.
- Hora en la que se ejecutó el proceso **STIME**.

```
profe@profe-vm:~$ ps -f
UID          PID  PPID  C  STIME TTY          TIME CMD
profe    2786  2778  0  16:34 pts/0    00:00:00 bash
profe   15299  2786  0  18:40 pts/0    00:00:00 ps -f
```

Comando: ps -e

En este caso, muestra los procesos en ejecución:

Nota acerca del carácter ?



ps - informa del estado de los procesos <<http://manpages.ubuntu.com/manpages/bionic/es/man1/ps.1.html>>

2.3.2. Comando pstree

El comando **pstree** visualiza, en forma de árbol, todos los procesos del sistema. Así podemos ver las relaciones que existen entre los procesos. Escribiendo **pstree --help** nos da las opciones más comunes. Es recomendable utilizarlo con la opción **-A** y **-G** para que te un árbol con líneas estilo ASCII y de terminal VT100 respectivamente, puedes añadir también **-u** para mostrar entre paréntesis al usuario propietario del proceso:

```
profe@profe-vm:~$ pstree -Au
systemd+-ModemManager---2*[{ModemManager}]
|-NetworkManager---2*[{NetworkManager}]
|-VGAuthService
|-systemd(profe)---(sd-pam)
|  |-at-spi-bus-laun+-dbus-daemon
|  |  `--3*[{at-spi-bus-laun}]
|  |-at-spi2-registr---2*[{at-spi2-registr}]
|  |-dbus-daemon
```

Mostrar las ramas del árbol de procesos de un usuario concreto con líneas de árbol ASCII:

```
profe@profe-vm:~$ pstree -A clase
gdm-x-session+-Xorg---{Xorg}
|-gnome-session-b+-ssh-agent
|  `--2*[{gnome-session-b}]
`--2*[{gdm-x-session}]

gnome-keyring-d---3*[{gnome-keyring-d}]

systemd+--(sd-pam)
|-at-spi-bus-laun+-dbus-daemon
|  `--3*[{at-spi-bus-laun}]
|-at-spi2-registr---2*[{at-spi2-registr}]
|-dbus-daemon
|-dconf-service---2*[{dconf-service}]
|-evolution-addre---5*[{evolution-addre}]
|-evolution-calen---8*[{evolution-calen}]
|-evolution-sourc---3*[{evolution-sourc}]
|-firefox+-Privileged Cont---18*[{Privileged Cont}]
|  |-2*[Web Content---18*[{Web Content}]]
```

2.3.3. Comando jobs

El comando **jobs** informa sobre los procesos que se están ejecutando tanto en primer plano como en segundo plano.

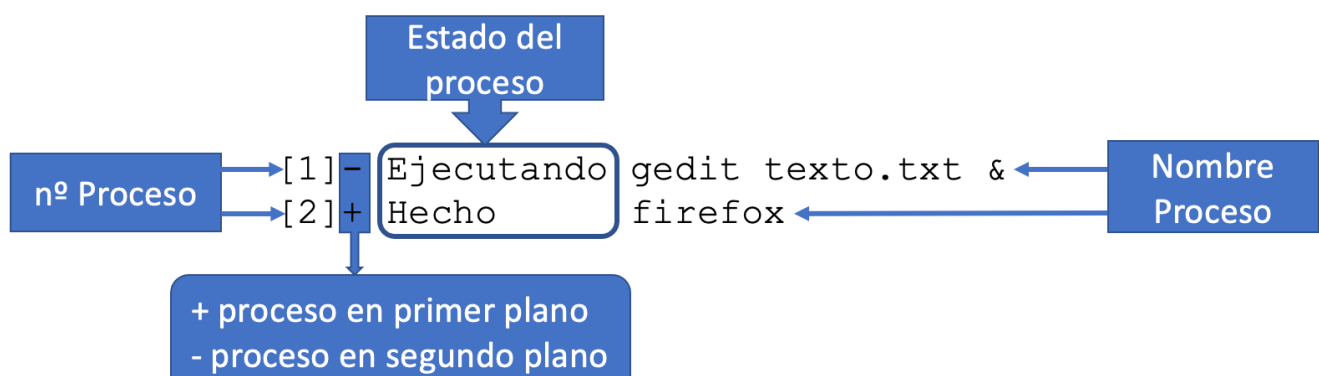
EJEMPLO

```
profe@profe-vm:~$ jobs
```

```
[1]- Ejecutando gedit texto.txt &
```

```
[2]+ Hecho firefox
```

Análisis de lo que el Comando jobs muestra por pantalla



2.3.4. Comando top

El comando `top` devuelve un listado de los procesos de forma parecida a como lo hace `ps`, con la diferencia que la información mostrada se va *actualizando periódicamente* lo que nos permite ver la evolución del estado de los procesos.

Además, en la parte superior muestra *información adicional*, como por ejemplo:

- El espacio en memoria ocupado por los procesos
- El espacio ocupado por la memoria de intercambio o swap
- El número total de tareas o procesos que se están ejecutando
- El número de usuarios o el porcentaje de uso del procesador.

En esta animación puedes apreciar como varía la información que muestra el comando `top`:

```
santos@santos-vm: ~
top - 20:26:52 up 3:53, 1 user, load average: 0,00, 0,00, 0,00
Tareas: 291 total, 1 ejecutar, 290 hibernar, 0 detener, 0 zombie
%Cpu(s): 0,7 usuario, 0,3 sist, 0,0 adecuado, 99,0 inact, 0,0 en espera, 0,0 hardw int, 0,0 softw int
MiB Mem : 3907,0 total, 389,0 libre, 1747,6 usado, 1770,4 búfer/caché
MiB Intercambio: 923,3 total, 923,3 libre, 0,0 usado. 1800,6 dispon Mem

  PID USUARIO  PR  NI  VIRT  RES  SHR S  %CPU  %MEM  HORA+  ORDEN
1779 santos    20   0 309940 76044 52080 S   1,0   1,9  0:20.81 Xorg
1904 santos    20   0 4080096 261936 102640 S   0,7   6,5  0:27.19 gnome-shell
2112 santos    20   0 296320 42856 30728 S   0,3   1,1  0:23.53 vmtotlsd
2778 santos    20   0 824712 54412 40684 S   0,3   1,4  0:08.69 gnome-terminal-
12678 santos   20   0 3237908 336224 170448 S   0,3   8,4  0:21.30 firefox
  1 root       20   0 103596 13000  8368 S   0,0   0,3  0:02.82 systemd
  2 root       20   0      0      0      0 S   0,0   0,0  0:00.01 kthreadd
  3 root        0 -20      0      0      0 I   0,0   0,0  0:00.00 rcu_gp
  4 root        0 -20      0      0      0 I   0,0   0,0  0:00.00 rcu_par_gp
  6 root        0 -20      0      0      0 I   0,0   0,0  0:00.00 kworker/0:0H-kblockd
  9 root        0 -20      0      0      0 I   0,0   0,0  0:00.00 mm_percpu_wq
10 root       20   0      0      0      0 S   0,0   0,0  0:00.13 ksoftirqd/0
11 root       20   0      0      0      0 I   0,0   0,0  0:01.62 rcu_sched
12 root       rt    0      0      0      0 S   0,0   0,0  0:00.18 migration/0
13 root      -51   0      0      0      0 S   0,0   0,0  0:00.00 idle_inject/0
14 root       20   0      0      0      0 S   0,0   0,0  0:00.00 cpuhp/0
15 root       20   0      0      0      0 S   0,0   0,0  0:00.00 cpuhp/1
16 root      -51   0      0      0      0 S   0,0   0,0  0:00.00 idle_inject/1
17 root       rt    0      0      0      0 S   0,0   0,0  0:00.33 migration/1
18 root       20   0      0      0      0 S   0,0   0,0  0:00.15 ksoftirqd/1
20 root        0 -20      0      0      0 I   0,0   0,0  0:00.00 kworker/1:0H-kblockd
21 root       20   0      0      0      0 S   0,0   0,0  0:00.00 kdevtmpfs
22 root        0 -20      0      0      0 I   0,0   0,0  0:00.00 netns
23 root       20   0      0      0      0 S   0,0   0,0  0:00.00 rcu_tasks_kthre
24 root       20   0      0      0      0 S   0,0   0,0  0:00.00 rcu_tasks_rude_
25 root       20   0      0      0      0 S   0,0   0,0  0:00.00 rcu_tasks_trace
```

2.4.- Comandos para cambiar el estado de los procesos

El comando **kill** permite enviar señales a los procesos para cambiar su estado. La señal por defecto de este comando es terminar (matar) el proceso, de ahí su nombre.



Jp.negre (Free Art License) <https://es.wikipedia.org/wiki/Archivo:1650_-_tercio_arquebuse2.jpg>

2.4.1. Comando kill

Genéricamente hablando, el comando **kill** envía una señal a un proceso. Para ver las diferentes señales que este comando puede enviar, hay que ejecutar el comando que se aprecia en la imagen siguiente:

```
santos@santos-vm:~$ kill -l
1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
6) SIGABRT     7) SIGBUS     8) SIGFPE     9) SIGKILL     10) SIGUSR1
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE    14) SIGALRM     15) SIGTERM
16) SIGSTKFLT  17) SIGCHLD    18) SIGCONT    19) SIGSTOP     20) SIGTSTP
21) SIGTTIN    22) SIGTTOU    23) SIGURG     24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF    28) SIGWINCH   29) SIGIO       30) SIGPWR
31) SIGSYS     34) SIGRTMIN   35) SIGRTMIN+1 36) SIGRTMIN+2 37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9 56) SIGRTMAX-8 57) SIGRTMAX-7
58) SIGRTMAX-6 59) SIGRTMAX-5 60) SIGRTMAX-4 61) SIGRTMAX-3 62) SIGRTMAX-2
63) SIGRTMAX-1 64) SIGRTMAX
```

Las principales señales que se utilizan con más frecuencia son:

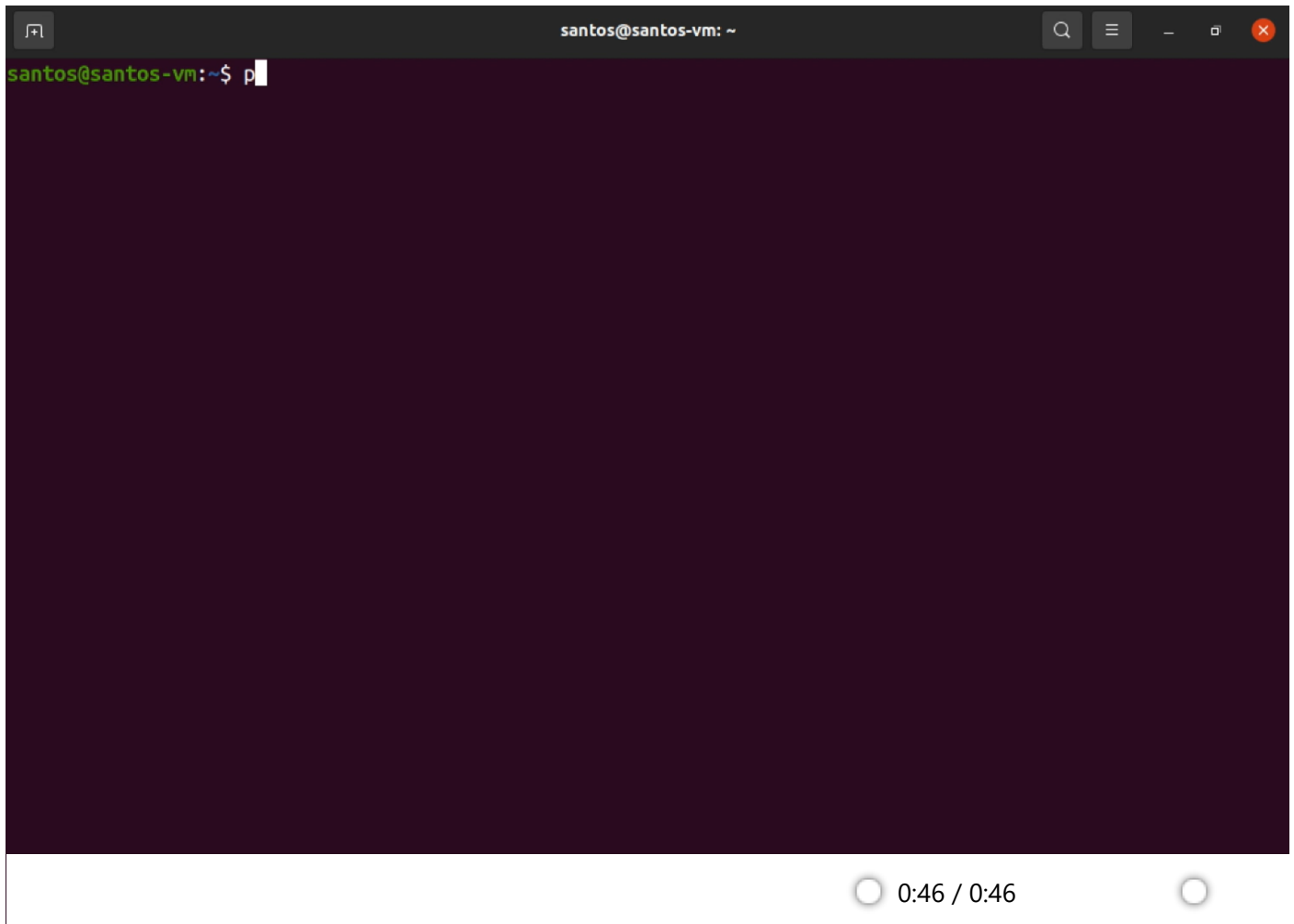
Señal	Significado	Señal	Significado
1.SIGHUP	Cuelgue del terminal o final detectado	2.SIGINT	Interrumpir el proceso. Atajo: CTRL+C
3.SIGQUIT	Salida del proceso. Atajo: CTRL+\	9.SIGKILL	Terminar el proceso (forzado)
15.SIGTERM*	Terminar el proceso (solicitado)	18.SIGCONT	Continuar la ejecución
19.SIGSTOP	Detener el proceso	20.SINGSTP	Como SIGSTOP. Atajo CTRL+Z

La señal por defecto* cuando no se indica una en concreto es la nº 15. SIGTERM.

Podemos enviar una señal utilizando el comando **kill** de dos maneras:

1. Utilizando el nombre de la señal.
2. Utilizando el número de la señal.

Fíjate en este vídeo sobre el uso de kill:



Debes saber que...

El comando **killall**, que funciona de manera similar a **kill**, pero con la diferencia de que, en vez de indicar un PID, se indica el nombre del programa, lo que afectará a todos los procesos que tengan ese nombre.

Por ejemplo si tenemos los procesos siguientes en ejecución:

```
$ ps
PID TTY TIME CMD
2786 pts/0 00:00:00 bash
19290 pts/0 00:00:02 firefox
19333 pts/0 00:00:00 Privileged Cont
19385 pts/0 00:00:00 WebExtensions
19428 pts/0 00:00:00 Web Content
19449 pts/0 00:00:00 Web Content
19478 pts/0 00:00:00 Web Content
19502 pts/0 00:00:00 ps
```

Los procesos con el nombre Web Content pertenecientes a diferentes pestañas del navegador Firefox, pueden ser eliminadas ejecutando:

```
$ killall -KILL "Web Content"
```

Si añadimos la opción **-i** al comando, nos pedirá confirmación para eliminar el proceso.

Si quieres ampliar la información sobre este comando visita el siguiente enlace:

Comando killall - Borrar procesos por su nombre <<http://manpages.ubuntu.com/manpages/focal/man1/killall.1.html>>



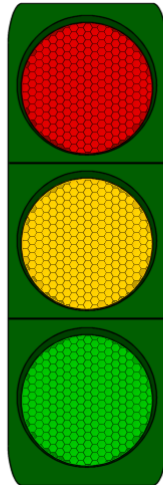
Para saber más

Para ampliar información sobre el comando kill y sus señales, visita el siguiente enlace:

Lista de señales del comando kill <<http://manpages.ubuntu.com/manpages/focal/es/man7/signal.7.html>>

2.5.- Comandos para cambiar la prioridad de los procesos

Los procesos en Linux tienen una prioridad determinada en su ejecución. Esta prioridad se establece en el inicio de ejecución del proceso o programa o, puede ser alterada, posteriormente. Para ello, haremos uso de los comando **nice** y **renice**.



Licencia: Dominio público

2.5.1. Comandos nice y renice

nice

El comando **nice** asigna una prioridad a un proceso antes de iniciarse. Con **nice** es posible iniciar un programa (proceso) con la prioridad modificada, más alta o más baja según se requiera.

Las prioridades van de -20 (la más alta) a 20 (la más baja). Solo root o el superusuario puede establecer *prioridades negativas que son más altas*.

Ejecutando el comando **ps -l** es posible observar la columna **NI** que muestra este valor. La sintaxis general del comando es:

nice -n prioridad proceso

Vamos a crear un proceso con prioridad 5 y, después, comprobamos la lista de procesos para confirmarlo:

```
santos@santos-vm:~$ nice -n 5 gedit
^Z
[2]+  Detenido                  nice -n 5 gedit
santos@santos-vm:~$ ps -l
```

	F	S	UID	PID	PPID	C	PRI	NI	ADDR	SZ	WCHAN	TTY	TIME	CMD
0	S	1000	2786	2778	0	80	0	-	3547	do_wai	pts/0		00:00:00	bash
4	S	1000	19290	2786	0	80	0	-	773330	poll_s	pts/0		00:00:05	firefox
4	S	1000	19333	19290	0	80	0	-	619566	poll_s	pts/0		00:00:01	Privileged Cont
4	S	1000	19385	19290	0	80	0	-	602578	poll_s	pts/0		00:00:00	WebExtensions
4	S	1000	19428	19290	0	80	0	-	610813	poll_s	pts/0		00:00:00	Web Content
4	S	1000	19449	19290	0	80	0	-	625237	poll_s	pts/0		00:00:01	Web Content
4	S	1000	19478	19290	0	80	0	-	598349	poll_s	pts/0		00:00:00	Web Content
0	T	1000	19677	2786	3	85	5	-	206819	do_sig	pts/0		00:00:00	gedit
4	R	1000	19684	2786	0	80	0	-	3529	-	pts/0		00:00:00	ps

Pulsa en la imagen para ampliar

renice

El comando **renice** cambia la prioridad de un proceso ya iniciado sin reiniciarlo. La sintaxis general del comando es:

renice prioridad PID

En el vídeo siguiente podemos ver el funcionamiento de este comando para cambiar la

prioridad a un proceso:

F

0:37 / 0:37



Para saber más

Para completar la información sobre estos comandos, accede al siguiente enlace:

Comando `nice` <<http://manpages.ubuntu.com/manpages/focal/en/man1/nice.1.html>>

Comando `renice` <<http://manpages.ubuntu.com/manpages/focal/en/man1/renice.1.html>>

2.6.- Comandos para cambiar el plano de los procesos

Cuando ejecutamos procesos, podemos ejecutarlos en primer plano o en segundo plano.

También veremos cómo llevar un proceso de primer plano a segundo plano y viceversa.

Linux nos permite gestionar los procesos para llevar los procesos de un plano a otro según nuestras necesidades. Esto lo logramos con comandos como **fg**, **bg** o **jobs**, como veremos a continuación.



Amateria1121 (CC BY-SA
<[http://
creativecommons.org/
licenses/?lang=es](http://creativecommons.org/licenses/?lang=es)>)

2.6.1. Creando procesos en segundo plano

nohup y &

Cuando se trata ejecutar procesos en background se utiliza el comando **nohup** o el operador **&**. Aunque realizan una función similar, no son lo mismo.

Un proceso iniciado en primer plano (foreground), monopoliza la terminal e impide que se inicien otros procesos desde la misma terminal hasta que este acaba. Por otra parte, un proceso en segundo plano no monopoliza la terminal y devuelve el control al usuario para seguir ejecutando programas.

Para iniciar un proceso en segundo plano se añade al final de la línea el carácter **&**:

```
$ gedit fichero.txt &
```

```
[1]  21612
```

En este ejemplo, vemos que el comando se ejecuta y devuelve un número de proceso [1] y un PID 21612, significa que lo ha enviado a segundo plano.

Los procesos ejecutados en una terminal, dependen de ella. Esto quiere decir que, independientemente de si están en primer plano o segundo plano, si cerramos la terminal los procesos creados desde la terminal también se cerrarán.

Para evitar esta situación, tenemos la posibilidad de ejecutar el comando **nohup**, que desliga el proceso creado desde la terminal a la misma terminal. Esto quiere decir que los procesos creados de esta manera, permanecerán en ejecución aunque se cierre la terminal.

Veamos un ejemplo del funcionamiento de este comando:



Recuerda que

Ejecutar un comando desde la terminal produce que el padre del proceso PPID que se invocó sea el proceso de la terminal en sí, por lo que si cerramos la terminal o salimos de la sesión también se terminarían los procesos hijos que dependan de la terminal y, para evitar esta situación, se usa el comando **nohup** que permite que quede inmune y al cerrar la aplicación siga en ejecución.

Si quieres saber más, pincha en el siguiente enlace:

comando **nohup** <<http://manpages.ubuntu.com/manpages/focal/en/man1/nohup.1posix.html>>

2.6.2. Cambiar de plano a los procesos



Ya sabemos que...

Ejecutar un comando añadiendo al final el carácter &, lo ejecuta y lo sitúa en segundo plano. Pero, ¿Cómo pasamos un proceso que estaba en segundo plano a primer plano? o al revés. Y, ¿Cómo vemos los trabajos activos?

Para mostrar en pantalla los trabajos actuales en ejecución, tenemos que escribir:

```
$ jobs
[1]- Ejecutando gedit saludo.txt &
[2]+ Detenido cat > hola.txt
```

Tenemos un trabajo en primer plano y otro en segundo plano. Si queremos que el trabajo [1] esté en primer plano, ejecutamos:

```
$ fg %1
$ jobs
[1]+ Ejecutando gedit saludo.txt &
[2]- Detenido cat > hola.txt
```

Para enviar un trabajo detenido a ejecutarse en segundo plano, podemos escribir:

```
$ bg %2
```



Para saber más

Puedes ampliar el conocimiento de estos comandos en los enlaces siguientes:

Comando `jobs` <<http://manpages.ubuntu.com/manpages/focal/en/man1/jobs.1posix.html>>

Comando `fg` <<http://manpages.ubuntu.com/manpages/focal/en/man1/fg.1posix.html>>

Comando `bg` <<http://manpages.ubuntu.com/manpages/focal/en/man1/bg.1posix.html>>

3.- Script de Linux



Caso práctico



Nuria Barroso Ruiz
(Elaboración propia)

Nora e Iván han estado probando Ubuntu en una máquina virtual y han decidido instalarlo en su equipo real, ya que cuando instaló Windows 11 se dejó una partición libre.

Ha aprovechado esta partición para ello. Ahora Nora tiene en el equipo de su casa dos sistemas operativos en la máquina real: Windows 11 y Ubuntu.

Cuando enciende el equipo le aparecen muchas opciones para arrancar el equipo, y si no selecciona ninguna, carga por defecto Ubuntu y el resto de la familia quiere que por defecto arranque Windows 11.

Con los conocimientos que ya tiene va a editar el gestor de arranque, que es un shell script que se ejecuta al iniciar el sistema y le va a modificar, entendiendo lo que hace el script en cada línea.

En Linux hay unos archivos, llamados script o guiones de la shell, que contienen órdenes o comandos que se van ejecutando. Algunos de estos shell script se ejecutan al iniciarse el sistema, como el gestor de arranque. Otros, los puede crear el propio usuario para mecanizar tareas que ejecute con frecuencia.

En este apartado vas a aprender a realizar pequeños shell scripts.

3.0. Ejecución de varios comandos

Vamos a ver unos conceptos importantes para cuando se quiere ejecutar varios comandos.

Ejecución secuencial

Como recordarás para la ejecución de varios comandos de forma secuencial usábamos el punto y coma ";"

```
comando1 ; comando2 ; ...; comandoN
```

Ejecución condicionada

Pero también podemos condicionar la ejecución de un comando a la ejecución o no de otro comando.

Ejecutar el comando2 solo si se ejecuta el comando1:

```
comando1 && comando2
```

Ejecutar el comando2 solo si no se ejecuta el primero

```
comando1 || comando2
```

3.1.- ¿Qué es un script?

Un script de comandos es un fichero de texto cuyas líneas son comandos ejecutables, tal y como las escribirías en la línea de comandos del shell. Los scripts pueden ejecutarse como si fueran un nuevo comando. Las líneas que empiezan con el carácter **#** son comentarios y son ignoradas por el shell al ejecutar el script.

El intérprete de comandos o "shell" es también un lenguaje de programación completo. La programación de shell se usa mucho para realizar tareas repetidas con frecuencia.

En Linux, cuando hagas un script, se *suele* comenzar con una línea que indica la shell con la que se va a ejecutar el script. Puede ser:

- `#!/bin/bash`
- `#!/bin/sh`
- `#!/bin/ksh`

Cualquier otra línea del script que comience por **#** es tomada como un comentario, por lo tanto, no será ejecutada. Los comentarios se añaden para facilitar la lectura del programa.

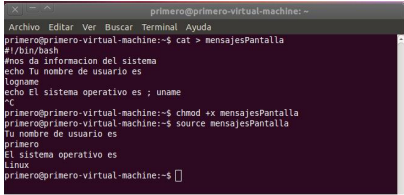
Para *comenzar a crear un script* tienes que:

- Escribir los comandos en un fichero de texto (con un editor de texto simple o redireccionando con `cat > nombre_script`). Por ejemplo, podemos utilizar el editor nano. En el terminal, escribiríamos la orden: `nano miScript.sh`. A partir de ese momento ya estaríamos en disposición de insertar las órdenes en el fichero.
- Escribir los comandos.
- Grabar o guardar.
- Cambiar los permisos del fichero para que tenga permiso de ejecución (con el comando `chmod`).
- Ejecutarle.

¿Cómo se ejecuta un script?

Escribir en la línea de comandos, el nombre del fichero de `miScript.sh`, como si fuera un nuevo comando. El shell busca el fichero en los directorios que están incluidos en la variable `PATH`. Si el directorio actual `'.'` no está en el `PATH`, para ejecutar el script hay que indicar donde se encuentra, por ejemplo: `./miScript.sh`

Normalmente, el directorio en donde se encuentra el script no se incluye en la variable



```
primero@primero-virtual-machine: ~  
$ cat > mensajesPantalla  
#!/bin/bash  
# Mensajes de informacion del sistema  
echo Tu nombre de usuario es  
logname  
echo El sistema operativo es ; uname  
-s  
primero@primero-virtual-machine:~$ chmod +x mensajesPantalla  
primero@primero-virtual-machine:~$ source mensajesPantalla  
Tu nombre de usuario es  
primero  
El sistema operativo es  
Linux  
primero@primero-virtual-machine:~$
```

Agustín Nieto Espino
(Elaboración propia)

PATH a no ser que se utilice mucho.

Para ejecutarle tienes varias opciones:

1. Dando el nombre de archivo como **argumento** a bash. Por ejemplo, para ejecutar el script `miScript.sh`, escribimos en el terminal la orden: `bash miScript.sh`
2. También se puede ejecutar escribiendo `sh` y a continuación el nombre del script. Por ejemplo: `sh miScript.sh`
3. Invocándolo **directamente** como un comando: `miScript.sh`.
4. Puede requerirse indicar una **vía absoluta o relativa**, o referirse al directorio actual como: `./miScript.sh`
5. Si el directorio actual **no está** contenido en la **variable PATH**: `source miScript.sh`

Las formas más usuales de ejecutar los script es escribiendo `./nombre_script` o bien escribiendo `sh nombre_script`.

Para que un fichero se pueda ejecutar en el sistema operativo Linux, sólo es necesario que tenga permisos de ejecución. Para poder ejecutarse no importa su nombre o extensión, sólo que tenga permisos de ejecución. Para asignar permisos de ejecución a un fichero debemos de utilizar el comando `chmod` como hemos visto en los apartados anteriores.



Autoevaluación

Para ejecutar un script:

- ☐ Con \$.
- ☐ Un script no se puede ejecutar.
- ☐ Con ./ nombre_script.
- ☐ Con #.

Incorrecta. Es un indicador del prompt.

No es correcta. Aunque es un fichero de texto, sí es ejecutable.

Muy bien, aunque hay más opciones que ésta.

No es cierto. Tiene otros significados.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta
4. Incorrecto



Debes conocer

A través del siguiente vídeo puedes ver cómo crear un script:

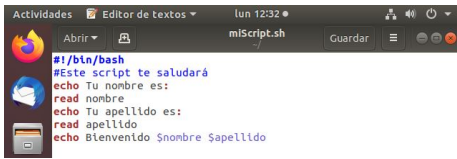
Creación de script



0:00 / 7:24

Agustín Nieto Espino. Descripción textual para el vídeo "Creación de script". (Elaboración propia)

3.2.- Variables



```
#!/bin/bash
#Este script te saludará
echo Tu nombre es:
read nombre
echo Tu apellido es:
read apellido
echo Bienvenido $nombre $apellido
```

Agustín Nieto Espino (Elaboración propia)

Las variables se pueden definir como un espacio en la memoria del sistema que puede contener un valor. Puede ocurrir que definamos una variable y ésta no contenga un valor hasta que no le sea asignado. Una variable de bash puede contener un **número**, un **carácter** o una **cadena de caracteres**. Las variables es un instrumento que encontraremos en los lenguajes de programación actuales.

En algunos de estos lenguajes a las variables es necesario definir las para posteriormente asignarle un valor. Cuando construyamos un script podemos utilizar variables, pero *no* necesitaremos definir las previamente.

Para ASIGNAR VARIABLES usamos el símbolo de igual =, o con el comando **set**. Por ejemplo, vamos a definir la variable número y le vamos a asignar el valor 5. Para ello, escribiríamos:

```
numero=5
```

Las variables **no se pueden acentuar** y **no es adecuado que comiencen por número o por algún carácter especial**.

A partir de este momento existirá una zona de memoria RAM reservada durante la ejecución del script. A esa zona de memoria se le identifica (o se accede) por su nombre y contendrá el valor 5.

Para obtener o RECUPERAR EL VALOR de la variable debemos precederla del símbolo \$, es decir, el carácter \$ nos permite expandir el valor de una variable dentro de una cadena.

Cuando escribimos la orden `set nombre`, lo que estamos haciendo es definir una variable llamada nombre cuyo valor va a ser solicitado al usuario en el momento de su ejecución. Cuando escribimos `$nombre` lo que estamos indicando es que en este lugar aparezca el valor almacenado en la variable nombre, su contenido.

Si asignamos a una variable algo entre paréntesis, lo que hacemos es CREAR UNA MATRIZ (conjunto) del número de elementos que hay entre paréntesis o entre comillas.

```
$colores = (rojo amarillo azul verde blanco)
```

Si quieres INTRODUCIR LOS VALORES a las variables desde la línea de comandos, deberemos solicitar dicha información al usuario utilizando el comando **read**. Este comando tiene como propósito solicitar información al usuario. Su ejecución captura las digitaciones realizadas, hasta obtener un carácter nueva línea (tecla **Enter** o **intro**).

Existen una serie de VARIABLES PREDEFINIDAS por el shell que contienen valores útiles. Por ejemplo:

- **\$HOME**, el nombre del directorio raíz del usuario.
- **\$PATH**, una lista de directorios separados por “:” donde el shell busca los comandos.
- **\$PWD**, el nombre del directorio actual.

El valor de cualquiera de ellas se puede ver en todo momento con el comando **echo**. Por ejemplo: **echo \$HOME**



Autoevaluación

Para introducir variables desde la terminal. ¿Usas el comando read?:

- ☐ Verdadero.
- ☐ Falso.

Correcto. Esta era sencilla.

Incorrecto. Para pedir desde el script que el usuario introduzca un dato, le guardo con el comando read.

Solución

1. Opción correcta
 2. Incorrecto
-

3.3.- Parámetros

Tienes que saber que los parámetros o argumentos son el conjunto de datos adicionales colocados detrás del nombre de una orden.

Quizá en ocasiones el usuario o usuaria desee crear un programa y ejecutarlo con diferentes conjuntos de datos.

Estos parámetros se introducen desde el símbolo del sistema, después del nombre del script. En la siguiente imagen puedes ver la orden de ejecutar el fichero `miscript.sh` al cual se le están pasando tres parámetros. *Los parámetros son datos adicionales que se le pasa al script para que sean utilizados durante la ejecución del script.* Cada parámetro o dato indicado en el momento de ejecutar un script se diferenciará uno de otro porque escribiremos un espacio después de cada uno de ellos exceptuando el último.



```
#!/bin/bash
#para ejecutarle escribimos
#source argumentos hola adios 3 4
echo el numero de argumentos pasados es: $#
echo El parametro 1 es: $1
echo El parametro 2 es: $2
echo El parametro 3 es: $3
echo el parametro 4 es: $4
```

Agustín Nieto Espino (Elaboración propia)



Agustín Nieto Espino (Elaboración propia)

Al ejecutar un script, se le pueden pasar parámetros o argumentos. Cada cadena (separada por espacios) que se le indique detrás del nombre del fichero en la línea de comandos es un parámetro (si se desea pasar una cadena que contiene espacios como un sólo argumento hay que encerrarla entre comillas).

Dentro del script, las variables `$1`, `$2`, `$3`, ... se substituyen por la cadena correspondiente a cada parámetro. Es decir, para recoger, en el momento de la ejecución del script los valores pasados por parámetros tenemos que utilizar el símbolo `$` seguido de la posición que ocupe el parámetro. Por ejemplo, si en el interior del fichero `miScript.sh` escribimos `$1`, en el momento de ejecutarse `$1` será substituido por el valor indicado. Si nos fijamos en la imagen anterior, será substituido por `valor1`.

Si el número de parámetro es igual o superior a 10, éste debe representarse utilizando {}. Por ejemplo, el parámetro 10 se referencia escribiendo `${10}`

Para referenciar a los parámetros desde el interior de un script, podemos utilizar las variables:

\$0: es la llamada al script. Su valor será igual al nombre del script que se esté ejecutando.

\$1: es el primer parámetro que se pasa.

\$2: es el segundo, etc.

\$#: es el número de parámetros que se pasan.

\$@ y \$*: son todos los parámetros, que se listan de una vez, en el orden que se han pasado.

\$\$: pid que tiene nuestro script. Número de procesos del shell.

\$?: es el valor de salida del último comando ejecutado. En un "exit 1", se devolvería el valor 1. Si todo sale bien, se devuelve el valor 0. Muy útil para saber si la última instrucción se ha ejecutado correctamente o no.

\$_: nos da el último argumento que hemos utilizado.

Ejemplo:

1. Script:

```
$ catparam.sh

#!/bin/bash
echo "Nombre:$0"
echo "Númerodeparámetros:$#"
echo "Parámetros:1=$12=$23=$3"
echo "Lista:$*"
echo "Elementos: $@"
```

2. Ejecución del Script:

```
$ param.sh juanito jorgito jaimito

Nombre:./param.sh
Número de parámetros:3
Parámetros: 1=juanito 2=jorgito 3=jaimito
Lista: juanito jorgito jaimito
Elementos: juanito jorgito jaimito
```

La diferencia entre **\$@** y **\$*** no salta a la vista. Retomamos el ejemplo anterior con una pequeña modificación:

```
$param.sh juanito "jorgito jaimito"
Nombre:./param.sh
Númerode parámetros:2
Parámetros: 1=juanito 2=jorgito jaimito 3=
Lista:juanito jorgito jaimito
Elementos: juanito jorgito jaimito
```

Esta vez sólo se pasan dos parámetros. Sin embargo, las listas parecen visualmente

idénticas. En realidad,

La primera contiene, en efecto:

"juanito jorgito jaimito"

La segunda contiene:

"juanito" "jorgito jaimito"

O sea, dos elementos. En el primer ejemplo tenía:

"juanito" "jorgito" "jaimito"



Autoevaluación

El parámetro \$2:

- ☐ Es el segundo parámetro.
- ☐ Es el primer parámetro.
- ☐ Es la llamada al script.

Muy bien. Has captado la idea

Incorrecta. El primer parámetro es \$1.

No es correcta. La llamada al script es \$0.

Solución

1. Opción correcta
 2. Incorrecto
 3. Incorrecto
-

3.4.- Comandos de programación

Algunos de los comandos que podemos emplear para la programación de scripts son **expr**, **true**, **false** y **test**.

El comando de programación: **expr** recibe números y operadores aritméticos como argumentos, efectúa los cálculos indicados y devuelve el resultado. Cada argumento debe estar separado por espacio blancos. Opera sólo con números enteros y realiza las operaciones de suma (+), resta (-), multiplicación (*), división entera (/), resto de división entera (%). Los símbolos * y / deben ser escapados escribiendo * y \/, al igual que los paréntesis, que deben escribirse \(y \).

El comando **expr** usa la **convención del lenguaje C** para cierto y falso: 0 es falso, y distinto de 0 es cierto. No confundir con la convención que toma el **shell** en sus valores true y false, que es la contraria.

Cuando utilicemos **expr** para que su resultado sea almacenado en una **variable**, debemos encerrar la expresión entre el símbolo ` (acento grave francés).

La orden **expr** también realiza operaciones lógicas de comparación, aceptando los operadores =, !=, >, <, >= y <=. El operador != es "no igual"; el ! se usa para negar.

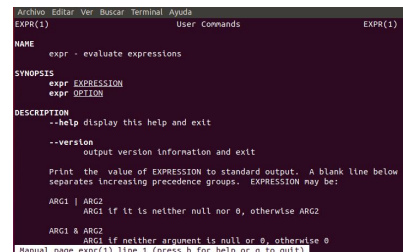
El comando **True**: Este comando no hace nada, sólo devuelve siempre 0, el valor verdadero. La ejecución correcta de un comando cualquiera devuelve 0.

El comando **False**: Este comando tampoco hace nada sólo devuelve siempre 1. Cualquier valor diferente de 0 se toma como falso. Las diversas condiciones de error de ejecución de los comandos devuelven valores diferentes de 0. Su significado es propio de cada comando.

El comando **Test**: Este comando prueba condiciones y devuelve valor cierto (0) o falso (distinto de 0) según el criterio de cierto y falso del shell. Esto lo hace apto para usar en la condición de if. Tiene dos formas equivalentes.

- **test expresión**
- **[expresión]**

Los espacios en blanco que hay entre la expresión y los corchetes son necesarios y debes ponerles siempre.



Agustín Nieto Espino
(Elaboración propia)



Autoevaluación

¿Es equivalente `test expresión o [ expresión ]`?

- ☐ Verdadero.
- ☐ Falso.

Correcta. Esta era sencilla, ¿verdad?

Incorrecto. Si son equivalentes. Prueba a escribir en la terminal `test -x fichero1` o bien `[ -x fichero1 ]` y comprueba que en ambos casos te testea si el fichero `fichero1` es ejecutable.

Solución

1. Opción correcta
 2. Incorrecto
-

3.5.- Operadores de comparación

= < >

Nuria barroso Ruiz
(Elaboración propia)

El comando u orden **test** nos permite llevar a cabo operaciones lógicas sencillas con ficheros, cadenas y números enteros. El formato de este comando es

test expresión

O de forma equivalente

[expresión]

La expresión puede tener cualquiera de los operadores que se muestran en las siguientes tablas:

Operadores de comparación aritméticos

-lt: menor estricto que	-le: menor o igual que	-eq: igual que
-gt: mayor estricto que	-ge: mayor o igual que	-ne: no igual que

Comprobación de atributos de fichero

Operador	Verdad (TRUE) si:
-d fichero	El fichero existe y es un directorio.
-e fichero	El fichero existe.
-f fichero	El fichero existe y es un fichero regular (no un directorio, u otro tipo)
-r fichero	Tienes permiso de lectura en fichero.
-s fichero	El fichero existe y no está vacío.
-w fichero	Tienes permiso de escritura en fichero.
-x fichero	Tienes permiso de ejecución en fichero (o de búsqueda si es un dir)
-O fichero	Eres el dueño del fichero.
-G fichero	El grupo del fichero es igual al tuyo.
fichero1 -nt fichero2	El fichero1 es más reciente que fichero2.
fichero1 -ot fichero2	El fichero1 es más antiguo que fichero2.

Comparaciones de cadenas alfanuméricas

Operador	Verdad (TRUE) si:
cadena1 = cadena2	cadena1 es igual a cadena2.
cadena1 != cadena2	cadena1 no es igual a cadena2.
cadena1 < cadena2	cadena1 es menor que cadena2.
cadena1 > cadena 2	cadena1 es mayor que cadena 2.
-n cadena1	cadena1 no es igual al valor nulo (longitud mayor que 0).

Operador	Verdad (TRUE) si:
-z cadena1	cadena1 tiene un valor nulo (longitud 0).

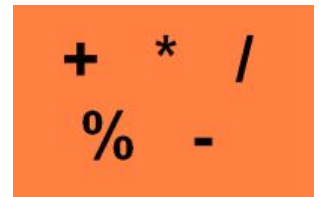
Estos operadores suelen emplearse junto con las estructuras condicionales que veremos más adelante.

3.6.- Operaciones aritméticas

Para realizar operaciones aritméticas se utiliza el comando **expr**, su sintaxis es: ``expr ARG1 OPERACION ARG2``

Donde *OPERACION* puede ser los siguientes símbolos:

- Para representar la suma utilizaremos el símbolo +
- Para representar la resta utilizaremos el símbolo -
- Para representar la división entera utilizaremos el símbolo / (barra invertida).
- Para representar la multiplicación utilizaremos el símbolo *
- Para representar el resto de la división utilizaremos el símbolo %



Nuria Barroso Ruiz
(Elaboración propia)

Prueba a hacer los siguientes ejemplos en la terminal:

```
expr 10 + 10
20
x=10
y=2
expr $x / $y
5
resta=`expr $x - $y`
mult=`expr $x * $y`
echo "$x menos $y vale $resta"
10 menos 2 vale 8
echo "La multiplicación de $x y $y es $mult"
La multiplicación de 10 y 2 es 20
```

DEBES SABER

Si el resultado del cálculo de una operación aritmética va a ser almacenado en una variable, se debe de definir encerrándola entre el símbolo ```, como en el ejemplo anterior.

Tienes varias formas de calcular valores dentro de un shell script. Para esto se utiliza una sintaxis especial, y es muy importante que los valores de las variables que se utilicen para hacer estos cálculos sean números únicamente. La sintaxis para hacer operaciones

aritméticas es la siguiente:

`$(expresión)` o bien `$((expresión))`

Las operaciones que puedes realizar son:

- suma `$((1+1))`
- resta `$((2-1))`
- multiplicación `$((2*2))`
- división `$((2/2))`
- otras: como suma de bits, obtener el módulo, evaluación de igualdad, etc.

Prueba esto en la línea de comandos (o en una shell): `echo 1 + 1`

Si esperabas ver '2', quedarás desilusionado o desilusionada. La solución es ésta: `echo $((1+1))`

Esto producirá una salida más 'lógica'. Esto se hace para evaluar una expresión aritmética. También puedes hacerlo de esta manera: `echo ${1+1}`

Ejemplo: Script que pide al usuario dos números y calcula las operaciones (suma, resta, multiplicación, división y cálculo del resto de la división entera) entre ellos:

```
#!/bin/bash

echo Introduzca un número:
read num1;
echo Introduzca un segundo número:
read num2;

resultado=`expr $num1 + $num2`;
echo El resultado de realizar la suma es $resultado;

resultado=`expr $num1 - $num2`;
echo El resultado de calcular la resta es $resultado;

resultado=`expr $num1 / $num2`;
echo El resultado de calcular la división es $resultado;

resultado=`expr $num1 % $num2`;
echo El resultado de calcular el resto de la división entera es $resultado;

resultado=`expr $num1 \* $num2`;
echo El resultado de calcular la multiplicación es $resultado;
```

Escapar operadores especiales

Es importante mencionar que al usar **expr** debes escapar los operadores especiales como el `*` con una barra invertida `\`, mientras que al usar los símbolos aritméticos convencionales no es necesario

3.7.- Depuración.

Al intentar ejecutar un script, te puede ocurrir que no se ejecute por algún error sintáctico. Para facilitarte la tarea de buscar el error está la **depuración**.

Se llama depuración ("**debug**") de un programa al proceso de verificar su funcionamiento en todos los casos posibles y corregir sus errores ("bugs", "pulgas"; del inglés, literalmente, "chinche"; por extensión, insecto pequeño).

Cuando se está escribiendo un script, puede convenir invocarlo de forma especial para generar información de comandos ejecutados y errores, para ayudar en la depuración. Las salidas se imprimen en el **error estándar**, por lo que pueden direccionarse a un archivo sin mezclarse con la salida del comando.

Veamos las dos opciones `-x` y `-v` del comando `bash`:

`bash -n nombre_del_script`

Ejecución en seco

`bash -x nombre_de_script`

Imprime cada comando en la salida.

traza: Expande cada orden simple e imprime por pantalla la orden con sus argumentos y a continuación su salida.

EJEMPLO 1 -x

Por ejemplo, si tengo el siguiente script llamado `mi_script1.sh`:

```
#!/bin/bash
name="Hanna Montana"
echo "Hello, $name"
```

Para ejecutar este script con depuración en la siguiente terminal, lo ejecutaríamos con: **`bash -x mi_script1.sh`**

Cuando se ejecute este script, mostrará el siguiente resultado:

```
+ name='Hanna Montana'
+ echo 'Hello, Hanna Montana'
```



bash -x

Nuria Barroso Ruiz
(Elaboración propia)

Hello, Hanna Montana

Si te fijas, antes de ejecutarse cada línea de tu script la imprime. Para distinguirlo incluye un **+** al comienzo de la línea.

EJEMPLO 2 -x

Para el caso de que el script incluya un bucle, al que hemos llamado `mi_script2.sh`, tenemos el siguiente ejemplo:

```
#!/bin/bash
for i in {1..5}; do
    echo $i
done
```

Cuando se ejecute este script, mostrará el siguiente resultado con: **bash -x mi_script2.sh**

```
+ echo 1
1
+ for i in {1..5}
+ echo 2
2
+ for i in {1..5}
+ echo 3
3
+ for i in {1..5}
+ echo 4
4
+ for i in {1..5}
+ echo 5
5
```

bash -v nombre_del_script

verbose: Invoca el script obteniendo información sobre cada comando ejecutado y su salida.

Imprime cada línea conforme la lee en el script, sin expandirla. De esta forma tu ves como escribiste cada una de las líneas en tu script y exactamente como es interpretada.

3.8.- Estructuras



Caso práctico



Nuria Barroso Ruiz
(Elaboración propia)

En la empresa de prácticas de Iván, le han pedido que, ya que se defiende con el entorno de texto de Ubuntu, que ayude en las tareas de administración del servidor de la empresa que es una distribución de Linux y sólo trabaja en entorno de texto.

Va a generar un script de manera que los usuarios se puedan dar de alta mediante un script y así, sólo tengan que introducir su usuario y contraseña, sin que tengan necesidad de saberse los comandos necesarios.

Iván va a tener que recordar las estructuras y funciones que se pueden definir desde la terminal.

Estructuras básicas de programación

Las estructuras básicas de programación son sólo dos: la estructura repetitiva y la estructura alternativa. Cada forma tiene sus variaciones, y la combinación de todas ellas generan múltiples posibilidades, pero detrás de cualquiera de ellas, por compleja que parezca, se encuentran siempre repeticiones o alternativas.

En este apartado vas a ver un conjunto de sentencias de control que soporta la bash Shell, que te van a permitir tomar decisiones o repetir ciertos grupos de comandos con facilidad. Las sentencias con las que vas a trabajar son:

- `if ...then...fi`
- `for...do...done`
- `while...do...done`
- `case...in...)... ) esac`
- `funciones.`

3.8.1. Estructuras condicionales IF

Las sentencias condicionales tienen muchas formas. La más básica es:

```
if expresión
    then
        sentencia
fi
```

Dónde sentencia sólo se ejecuta si la expresión se evalúa como verdadera.

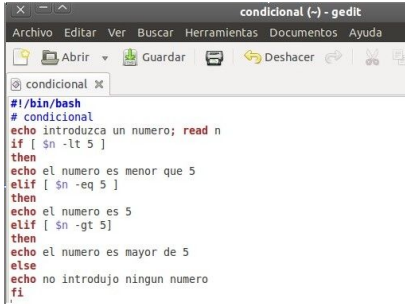
Los condicionales tienen otras formas compuestas, como:

```
if expresión
    then
        sentencia1
    else
        sentencia2.
fi
```

Aquí sentencia1 se ejecuta si la expresión es verdadera. De otra forma se ejecuta sentencia2.

Otra forma más de condicional es:

```
if expresión1
    then
        sentencia1
    elif expresión2
    then
        sentencia2
else
    sentencia3.
```

A screenshot of a terminal window titled 'condicional (-) - gedit'. The window shows a bash script that prompts the user to enter a number and then uses an if-then-elif-then-else-fi structure to check the number and print a message. The script is as follows:

```
#!/bin/bash
# condicional
echo introduzca un numero; read n
if [ $n -lt 5 ]
then
echo el numero es menor que 5
elif [ $n -eq 5 ]
then
echo el numero es 5
elif [ $n -gt 5]
then
echo el numero es mayor de 5
else
echo no introdujo ningun numero
fi
```

Agustín Nieto Espino
(Elaboración propia)

fi

En esta forma sólo se añade `elif expresion2 then sentencia2`, que hace que `sentencia2` se ejecute si `expresión2` se evalúa verdadera.

El comando `if` implementa una estructura alternativa. Su sintaxis es:

```
if expresión ; then comandos1 ; [else comandos2 ;] fi
```

equivalente a:

```
if expresión  
then
```

```
    comandos1
```

```
[else
```

```
    comandos2]
```

```
fi
```

Construcciones más complejas pueden hacerse usando `elif` para anidar alternativas.

El texto que aparece entre corchetes es opcional. No es obligatorio ponerlo siempre.

Para practicar puedes intentar ejecutar el siguiente script:

```
#!/bin/bash  
if [ -w /etc/passwd ]  
then  
    echo "Tengo permisos de escritura en el archivo /etc/passwd"  
else  
    echo "NO tengo permisos de escritura en el archivo /etc/passwd"  
fi
```



Autoevaluación

Con la sentencia IF cuando comparas dos cadenas, ¿es necesario el ELSE?

☐ Verdadero.

☐ Falso.

Correcta. Esta era sencilla, ¿verdad?

Incorrecto. Hace falta, ya que nos dice que son dos sentencias a testear. Estas irán separadas por el ELSE.

Solución

1. Opción correcta

2. Incorrecto

3.8.2. Bucle o estructura FOR

Este comando le usarás para implementar una estructura repetitiva, en la cual, una secuencia de comandos se ejecuta una y otra vez.

Su sintaxis es:

```
for variable in lista ;  
  
do  
  
    comandos  
  
done
```

Cada una de las órdenes o comandos que indiquemos que se repita dentro del bucle o estructura **for** deberá de terminar con el símbolo ";" a no ser que se trate de la definición de una estructura.

El bucle **for**, necesita de una *variable* para su uso junto con un conjunto de valores. En cada vuelta que da el bucle, la variable tendrá asignado uno de los valores de la lista. Por lo tanto, el bucle o estructura repetitiva **for**, *dará tantas vueltas como posibles valores puedan ser asignado a la variable*.

Ejemplos: Se puede probar en la línea de comandos:

```
for NUMERO in 1 2 3 4 ; do echo $NUMERO ; done
```

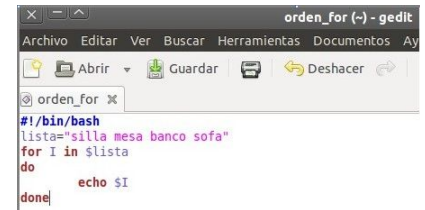
```
for NOMBRE in alfa beta gamma ; do echo $NOMBRE ; done
```

```
for ARCH in * ; do echo Nombre archivo $ARCH ; done
```

El carácter ***** es expandido por el shell colocando en su lugar todos los nombres de archivos o subdirectorios del directorio actual.

Puedes probar a ejecutar el siguiente ejemplo:

```
for LETRA in a b c d e f;  
do  
echo $LETRA  
done
```



Agustín Nieto Espino (Elaboración propia)

Verás que te muestra las letras comprendidas desde la “a” hasta la “f”.

La variable del “for”, irá tomando cada uno de los valores especificados después del in en cada bucle o vuelta.

Otro ejemplo clásico, es que la variable tome en cada bucle, el valor de un parámetro proporcionado en la línea de comandos.

```
for PARAMETRO in $*;
do
    echo $PARAMETRO
done
```

Mostrará cada uno de los parámetros tecleados en la línea de comando.

Otro ejemplo, variando la lista es:

```
for i in `seq 0 9`;
do

    echo $i

done
```

En el ejemplo anterior se muestra en el terminal los números del cero al nueve. Por lo tanto, la variable i va a tomar el valor 0 en la primera vuelta, va a tomar el valor 1 en la segunda vuelta y así hasta llegar a 9. El conjunto de los números entre el cero y el nueve se genera con el comando `seq 0 9`. Debes de prestar atención al entrecomillado.

La lista cuando es un conjunto largo, la puedes expresar así: (valor inicial; valor final; como se incrementa). Un ejemplo típico es la tabla de multiplicar:

```
#!/bin/bash
echo Introduce un numero para hacer su tabla de multiplicar:
read n
for ((i=0;i<=10;i++)); do

    echo $n * $i es igual a ${n*i}

done
```

La definición del **for** incluye:

- el valor inicial que va a tomar la variable i
- la condición que se va a cumplir para que se repita el bucle
- el incremento de la variable i para que alcance un valor que haga que la condición deje de cumplirse

Si utilizamos esta última definición, tendremos que ejecutar el script utilizando el comando bash seguido del nombre del fichero, o bien, escribiendo ./nombrescript.



Autoevaluación

Con la orden FOR, ¿es necesario el IN?

- ☐ Verdadero.
- ☐ Falso.

Correcta. Esta era sencilla, ¿verdad?

Incorrecta. Con la orden FOR, necesitamos IN para indicar en qué cadena hay que hacer el recorrido.

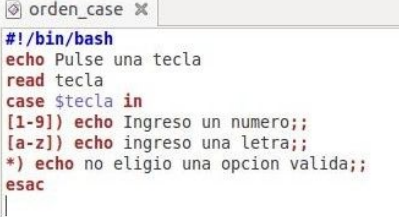
Solución

1. Opción correcta
 2. Incorrecto
-

3.8.3. La estructura CASE

Este comando implementa alternativas o "casos". Elige entre múltiples secuencias de comandos, la secuencia a ejecutar. La elección la realiza encontrando el primer patrón con el que coincida una cadena de caracteres. Su sintaxis es:

```
case expresión in
patrón1) comandos1;;
patrón2) comandos2;;
...
*) comandosN;;
esac
```



```
#!/bin/bash
echo Pulse una tecla
read tecla
case $tecla in
[1-9]) echo Ingreso un numero;;
[a-z]) echo ingreso una letra;;
*) echo no eligio una opcion valida;;
esac
```

Agustín Nieto Espino
(Elaboración propia)

El patrón `*)` se coloca al final, representa el patrón *por defecto*. Sus comandos se ejecutan en el caso de que expresión no coincida con ninguno de los patrones precedentes, es decir, cuando ninguna de las opciones anteriores fue satisfecha.

La estructura case la usarás normalmente para generar menús y poder elegir distintas opciones. Puedes intentar hacer el siguiente ejemplo:

```
#!/bin/bash
echo Elige alguna de las siguientes opciones:
echo 1 Tu directorio de trabajo es:
echo 2 El usuario es:
echo 3 Salir
read opcion
case $opcion in
1) pwd ;;
2) whoami;;
3) exit;;
*) echo No has elegido ninguna opción propuesta.;;
esac
```

Observa que se debe de indicar cada uno de los valores que puede ser seleccionado seguido de un cierre de paréntesis. En el ejemplo anterior, los valores que puede seleccionar el usuario son 1, 2, 3 u otro. Vemos que aparece "1), 2) y 3)", para indicar el comienzo de cada opción.

Muy importante no olvidar que cada opción de la estructura case termina con dos puntos y comas ";;"

Dentro de cada opción podemos indicar tantas instrucciones como necesitemos. Cada una de ellas, terminará con punto y coma ";" exceptuando la última instrucción de la opción que terminará con dos puntos y comas ";;"

Cuando queramos que una opción pueda tomar un conjunto de valores diferentes, por ejemplo, si el usuario pulsa un número comprendido entre uno y nueve, deberemos definir el rango de valores que se le puede asignar a la opción encerrados entre corchetes e indicando el comienzo y el fin del rango separados por un guion. Por ejemplo, para definir un rango entre 1 y 9, escribiríamos: [1-9]



Autoevaluación

Con la estructura CASE, ¿se generan menús?

- ☐ Verdadero.
- ☐ Falso.

Correcta. Esta era sencilla, ¿verdad?...

Incorrecta. Creo que te falta poner más atención. Despéjate primero, e inténtalo luego.

Solución

1. Opción correcta
 2. Incorrecto
-

3.8.4. Bucle o estructura WHILE

Este comando implementa una estructura repetitiva, en la cual, el conjunto de comandos se ejecuta *mientras* se mantenga válida una condición. La condición se examina al principio y luego cada vez que se completa la secuencia de comandos. Si la condición es falsa desde la primera vez, los comandos no se ejecutan nunca.

Su sintaxis es:

```
while condición; do

    comandos

done
```

La construcción **while** (en inglés, significa mientras) es adecuada en los casos en los que hay que evaluar *cuándo* hacer algo.

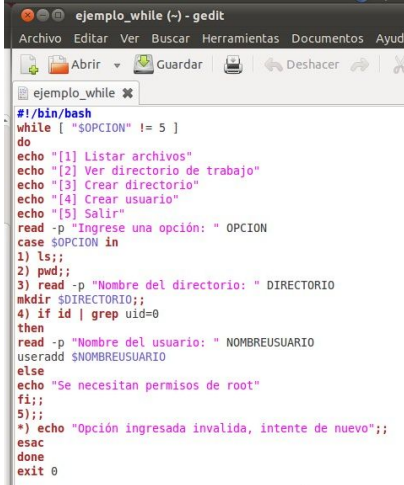
En cambio, el uso del **for** sigue el patrón de repetirse mientras se cumpla una condición. Prueba a intentar el siguiente ejemplo:

```
while [ $EN != "S" ]
do

    read -p "Escribe S para salir: "

done
```

En el ejemplo anterior podemos ver la sencilla sintaxis que tiene el **while**; el ciclo no terminará hasta que el usuario escriba la letra "S" (debe ser mayúscula).



```
#!/bin/bash
while [ "$OPCION" != 5 ]
do
    echo "[1] Listar archivos"
    echo "[2] Ver directorio de trabajo"
    echo "[3] Crear directorio"
    echo "[4] Crear usuario"
    echo "[5] Salir"
    read -p "Ingrese una opción: " OPCION
    case $OPCION in
        1) ls;;
        2) pwd;;
        3) read -p "Nombre del directorio: " DIRECTORIO
            mkdir $DIRECTORIO;;
        4) if id | grep uid=0
            then
                read -p "Nombre del usuario: " NOMBREUSUARIO
                useradd $NOMBREUSUARIO
            else
                echo "Se necesitan permisos de root"
            fi;;
        5) ;;
        *) echo "Opción ingresada inválida, intente de nuevo";;
    esac
done
exit 0
```

Agustín Nieto Espino
(Elaboración propia)



Debes conocer

En el siguiente enlace puedes ver ejercicios resueltos:

Ejemplos de scripts. <<https://asanzdiego.github.io/curso-nivelacion-bigdata-2017/ejercicios/ejercicios-shell-script-solucion.pdf>>



Autoevaluación

Relaciona el comando con una breve descripción, escribiendo el número asociado a lo que hace el comando en el hueco correspondiente.

Ejercicio de relacionar

Símbolo	Relación	Significado
If	☐	1. Se complementa con el do y done.
for	☐	2. Estructura condicional.
while	☐	3. Necesitas el IN.
Case	☐	4. Proporciona un menú.

Son órdenes empleadas en los script.

3.9.- Funciones

```
funciones %  
#!/bin/bash  
echo Este script contiene una funcion  
function area()  
{  
  echo introduce un lado  
  read lado  
  resultado1=$((lado*lado))  
  echo el area es $resultado1  
}  
function perimetro()  
{  
  echo introduce un lado  
  read lado  
  resultado2=$((4*lado))  
  echo el perimetro es $resultado2  
}  
echo Se va a calcular el area  
area  
echo el perimetro  
perimetro
```

Agustín Nieto Espino

(Elaboración propia)

Las funciones en los scripts te serán necesarias cuando una tarea que se realiza repetidamente dentro del mismo programa puede *escribirse aparte e invocarse* como una "función". Las funciones agrupan instrucciones que se van a utilizar muy a menudo dentro de un programa y para evitar tener que repetirlas continuamente se agrupan bajo una función. De tal forma, que cuando estamos construyendo el script, en lugar de escribir una y otra vez las mismas instrucciones se hace una llamada a la función. De esta forma, al ejecutarse el script, en donde aparece la llamada a la función, se sustituye por las instrucciones definidas dentro de la función.

Para definir una función es preciso elegir un nombre y escribir un trozo de código asociado a ese nombre. La función puede recibir algún valor como "parámetro" en base al cual realiza su tarea. Definida así la función, para introducirla en el script, basta escribir su nombre y colocar el valor del parámetro entre paréntesis.

Una función o subrutina es una agrupación de comandos bajo un mismo nombre. Esto nos permitirá dividir el programa en bloques y programar cada bloque por separado, o también agrupar una serie de comandos que se ejecutan a menudo, bajo un mismo nombre.



Autoevaluación

Para invocar a una función en un script, ¿es necesario simplemente escribir su nombre?

- ☐ Verdadero.
- ☐ Falso.

Correcta. Esta era sencilla, ¿verdad?

Incorrecta. En el script cuando llamamos a una función, basta escribir el nombre de la función.

Solución

1. Opción correcta
2. Incorrecto



Para saber más

En el siguiente enlace puedes ver un resumen de los comandos que puedes utilizar a través del terminal: [Resumen de comandos <https://www.lawebdelprogramador.com/pdf/files/1501149403_resumcom.jpg?1521629505.6507>](https://www.lawebdelprogramador.com/pdf/files/1501149403_resumcom.jpg?1521629505.6507)

4.- Automatizar tareas

Cuando realizamos tareas de administración hay ocasiones en que queremos ejecutar programas o comandos en un determinado momento, o de forma periódica, para ahorrar tiempo y esfuerzo podemos indicarle al sistema que ejecute estas tareas por nosotros en un momento determinado. Bastará editar un archivo o un simple comando y nuestro Linux se ocupará de todo.

Esto lo podemos hacer mediante el comando **at** o mediante el servicio **crontab**.

4.1. Crontab y cron

Crontab ejecuta un demonio, **Cron**, que se ejecuta una vez por minuto, y decide si debe ejecutar las acciones que le hemos indicado según lo que le hayamos especificado en el archivo `/etc/crontab`. Cada usuario guardara su propia versión de este archivo.

Cron es un demonio (que es como se conoce a un proceso en segundo plano) que se ejecuta desde el mismo instante en el que arranca el sistema operativo. Cron se encargará de comprobar si existe alguna tarea (job) para ser ejecutada, de acuerdo a la hora configurada en el propio sistema operativo. Es muy importante que la hora esté bien configurada, y también la zona horaria, de lo contrario, las ejecuciones que realice Cron no se corresponderán con nuestras configuraciones.

Para garantizar que nuestro sistema operativo esté configurado correctamente, es muy importante que obtenga la hora automáticamente de los servidores NTP (Network Time Protocol) que existen.

Para comprobar que tenemos la hora correcta en nuestro sistema operativo, en qué zona horaria estamos y si el reloj del sistema está correctamente sincronizado con los servidores NTP, podemos ejecutar la siguiente orden:

```
timedatectl
```

Si todas las horas cuadran perfectamente y el tiempo es el correcto, esto es muy importante, de lo contrario, los «Cron» se ejecutarán en horas donde no deberían ejecutarse. Habría un problema si el sistema está mal configurado y coge una hora que no corresponde. En el caso de que no tengamos la zona horaria bien configurada, podremos configurarla de forma correcta con la siguiente orden:

```
timedatectl set-timezone Europe/Madrid
```

En función de la distribución, Cron se inicia utilizando las carpetas `/etc/rc.d/` o `/etc/init.d` y cada minuto comprueba los ficheros `/etc/crontab` o `/var/spool/cron` en busca de posibles ejecuciones que hayamos programado. Con esto podremos automatizar muchos procesos, agregando nuevos comandos o scripts con tareas a cron. Nos resultará muy útil, por ejemplo, para automatizar actualizaciones de los sistemas o para tener un protocolo de copias de seguridad. Pero éstas deben estar siempre escritas en un fichero Crontab.

Puede tener diferentes valores, según sea el periodo de tiempo en el que va a ejecutar una

acción. Por ejemplo hacer algo cada hora, cada día, cada semana, siempre a un momento concreto del día, etc. Todo esto lo vas a poder configurar y programar para evitar que algo no se ejecute correctamente. Lo que va a hacer es revisarlo siempre que llegue el momento exacto.

Para comprobar que el demonio está activo podemos ejecutar la orden:

```
systemctl status crond
```

Si está activo: q

si no lo está, para activarlo habrá que ejecutar: `systemctl start crond`

4.1.1. Crontab

Crontab es un archivo de texto, se trata de un archivo con un contenido especial y específicamente diseñado para que sea leído correctamente por Cron y proceder con la ejecución que nosotros hayamos programado. Crontab posee una lista con todos los scripts a ejecutar, generalmente cada usuario del sistema posee su propio fichero Crontab, de esta forma, cada usuario podría programar sus propias tareas repetitivas independientemente, sin necesidad de que siempre tengamos que acudir al usuario administrador.

Para generar el archivo propio, cada usuario deberá hacer uso del comando «crontab» (sí, es el mismo nombre que la propia herramienta). Se utiliza mucho para poder ejecutar y administrar tareas de Cron en un sistema.

Con Crontab, como vas a ver, podrás configurar el archivo según unos parámetros establecidos. Podrás poner un tiempo exacto para que ejecute la acción (o múltiples tareas). Su nombre, como en el caso de Cronos, viene de la palabra griega «tiempo», por lo que esa va a ser la clave de su funcionamiento.



Debes conocer

Además de lo anterior, hay que tener en cuenta que la sintaxis crontab cuenta con hasta cinco campos. Cada uno de estos tiene una serie de valores que puedes encontrar en las siguientes líneas:

Minuto: valores 0-59.

Hora: valores 0-23.

Día del mes: valores 1-31.

Mes: valores 1-12.

Día de la semana: valores 0-6. El 0 representa el domingo. Ten en cuenta que en algunos sistemas, el valor 7 representa el domingo.

Comando: valores Orden a ejecutar.

En la ayuda del comando crontab -e, nos aparece:

🎯 A crontab file has five fields for specifying:

```
* * * * * command to be executed
- - - - -
| | | | |
| | | | +----- **DAY OF WEEK** (0-6) (Sunday=0)
| | | +----- **MONTH** (1-12)
| | +----- **DAY OF MONTH** (1-31)
| +----- **HOUR** (0-23)
+--- **MINUTE** (0-59)
```

5 asteriscos y el comando a ejecutar. Cada uno de los 5 asteriscos significa:

m: minuto

h: hora

dom: día del mes

mon: mes

dow: día de la semana

Los valores que pueden adoptar cada una de estas variables se encuentran en la imagen. Por ejemplo, en el caso del último, se permite la utilización del nombre de los días escritos en inglés. Aunque en la imagen anterior no aparezca, es necesario indicar entre el comando y el último asterisco (el día) el propietario del archivo para que funcione correctamente.

Para editar el archivo existente en la actualidad se utiliza el comando visto: `crontab -e`

Ejemplos:

Si queremos ejecutar el script de consulta.sh todos los días a las 7 de la tarde, tendremos que indicar una línea en el crontab con lo siguiente:

```
00 19 * * * usuario /ubicacion/del/script/consulta.sh
```

Si queremos ejecutar todos los domingos a las 7 de la tarde:

```
00 19 * * 0 usuario /ubicacion/del/script/consulta.sh
```

Para ejecutar el script todos los 4 de febrero a las 7 de la tarde:

```
00 19 4 2 * usuario /ubicacion/del/script/consulta.sh
```



Otras opciones

Listar todas las tareas existentes en el crontab del usuario:

```
crontab -l
```

Borrar el crontab que está configurado:

```
crontab -d
```



Para saber más

Existen una serie de cadenas de texto que ya vienen configuradas de manera predeterminada para que se puedan programar scripts de forma automática. Estas son las más comunes:

@reboot: este sirve para que se ejecute el script una vez que se inicia el equipo.

@yearly o @annually: consigue que se ejecute el script una vez al año.

@monthly: para que se ejecute de manera mensual, el primer día de cada mes.

@weekly: en este caso se puede usar para que se ejecute una vez por semana.

@daily: para todos los días.

@midnight: en este caso se daría todas las noches a las doce.

@hourly: esta cadena servirá para que se dé durante el primer minuto de cada hora del día.

En resumen, aquí tienes los PASOS PARA AUTOMATIZAR TAREAS:

1.- Comprobar que el demonio está activo

2.- Comprobar hora del sistema

3.- Crear el script y darle permiso de ejecución

4.- Programar fichero crontab

4.2. Comando at

El comando **at** permite programar tareas únicas en nuestro sistemas GNU/Linux.

No hace falta decir que para programar tareas repetitivas debemos utilizar cron y crontab, de los que ya hemos hablado en el apartado anterior.

Con at podemos leer comandos de entrada estándar o scripts que deseamos que se ejecuten más tarde, una única vez.

El comando at, nos puede ser útil para apagar el sistema a una hora específica, realizar una copia de seguridad única, enviar un correo electrónico como recordatorio a la hora especificada, entre otras muchas cosas.



Opciones del comando at

- **at:** ejecuta comandos a la hora especificada.
 - **atq:** enumera los trabajos pendientes de los usuarios. (alias `at -l`)
 - **atrm:** borra trabajos por su número 1.de trabajo. (alias `at -r`)
 - **at -c <num_tarea>:** para ver el contenido de una tarea en concreto
-

Sintaxis de at

```
at hora [fecha]
```

Cómo crear una tarea única con at

Los pasos serían los siguientes:

- 1) Ejecutar el comando at indicando hora y día (opcional) en que se desee se ejecute la tarea.
- 2) Escribir cada comando a ejecutar en una línea, pulsar INTRO para finalizar la entrada de cada comando.
- 3) Finalizar la entrada de la tarea at, con CTRL+D

Cosas importantes a tener en cuenta:

- a) Los comandos se ejecutan con `/bin/sh` no con `/bin/bash`
- b) Si la hora de ejecución ya pasó la tarea no se ejecutará

Todas las tareas se almacenan en el directorio: `/var/spool/cron/atjobs`

Ejemplo de ejecución del comando at

```
at 11:40 2024-02-13
```

```
ls -ltr /tmp > "~/prueba_comando_at.txt"
```

```
echo 'finalizado ' >> "~/prueba_comando_at.txt"
```

```
CTRL + D
```

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0
<<http://creativecommons.org/licenses/by-sa/4.0/>>